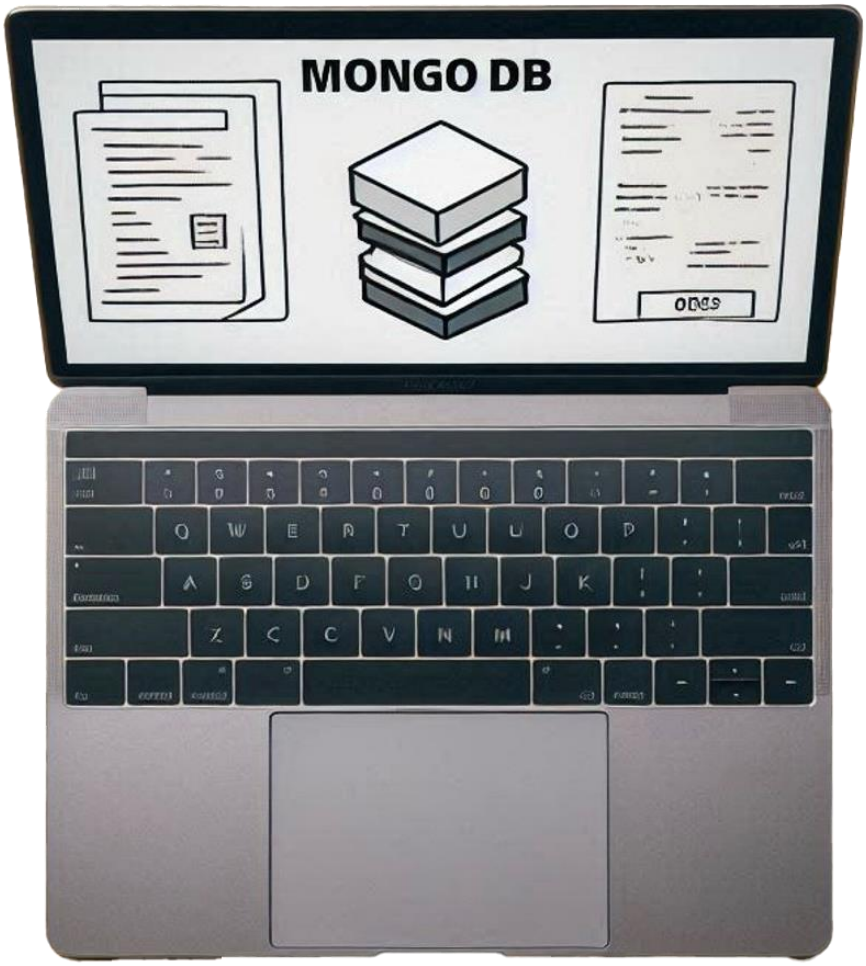



# Bases de datos documentales

# MongoDB

Pablo Ruipérez - 2025

# Índice de contenido


BASES DE DATOS DOCUMENTALES 1

MONGODB 1

ÍNDICE DE CONTENIDO 2

INTRODUCCIÓN A BASES DE DATOS DOCUMENTALES 4

**Bases de Datos Documentales** **4**
Ejemplos de documentos: 4

MONGODB 8

**Introducción** **8**

**Aplicaciones de MongoDB** **8**

**Modelo de datos de MongoDB** **10**
Documentos en Formato BSON 10

**Tipos de datos en MongoDB** **11**
Ejemplo: 12
Más tipos de datos en MongoDB 13

**Conceptos Básicos del Modelo de Datos de MongoDB** **14**
Ejemplo de colección de documentos 16

**Factores Importantes en el Modelo de Datos de MongoDB** **17**
Consideraciones al Diseñar con MongoDB 18

**Ejemplo de base de datos: solución relacional y soluciones MongoDB** **18**
Solución con base de datos relacional 18
Soluciones con MongoDB 19

**Modelo de datos de MongoDB, relaciones entre los datos** **21**
One-to-One (Uno a Uno) 21
One-to-Many (Uno a Muchos Embebido) 22
One-to-Many (Uno a Muchos con Referencias) 22
Many-to-Many (Varios a Varios) 23
Decisión sobre el modelo de datos a emplear en MongoDB 24

**Operaciones básicas en MongoDB** **25**
Creación de la Base de Datos 25
Utilización de la Base de Datos 25
Borrado de la Base de Datos 25
Creación de Colecciones 25

**Operaciones CRUD en MongoDB** **27**
Crear datos - *Insert* 27
Actualización de datos - `update` 29
Borrado de datos – `delete` y `drop` 30

CONSULTAS EN MONGODB 31

**Introducción** **31**

**Método `find()`** **31**

Introducción 31
Primeros pasos 32
Sintaxis general del comando `find()` 34
Condiciones - operadores de comparación 34
Condiciones - operadores lógicos 36
Condiciones – Anidamiento 38
Condiciones – Existencia de campo ($exist) 40
Condiciones – Tipo de un campo ($type) 41
Condiciones – Sobre arrays ($all, $elemMatch, $size) 41
Modificadores 43

AGREGACIONES 45

**Introducción 45**
¿Qué son las agregaciones? 45
Etapas de una agregación 45
`find()` o `aggregate()` 46
Etapa `$match` 47
Etapa `$group` 48
Operaciones de agregación 49
Etapa `$sort` 52
Etapa `$limit` 53
Etapa `$project` 55
Etapa `$count` 56
Etapa `$set` 57
Etapa `$skip` 59
Etapa `$unwind` 60
Etapa `$lookup` 63
Etapa `$out` 64
Uso de `$project` y `$set` 66

ÍNDICES 69

**Introducción 69**

**Tipos de índices 70**
Índice en el campo _id 70
Índice simple 71
Índice compuesto 72
Índice multikey 75

**Opciones de Índices 76**
`unique` - restricción de unicidad 76
`sparse`: Índice solo sobre documentos con el campo presente 76
`expireAfterSeconds`: Índice con tiempo de vida (TTL) 77
`partialFilterExpression`: Índice sobre documentos que cumplen una condición 78
`background`: Creación de índices sin bloquear la base de datos 79
`collation`: Índices con reglas específicas de ordenamiento 79

**Índices sobre texto 80**

**Consideraciones finales sobre índices en MongoDB 82**
Cuándo crear índices 82
Otros tipos de índices en MongoDB 82

**Más sobre índices 84**

# Introducción a Bases de Datos Documentales

En el mundo actual, donde los datos se generan a un ritmo sin precedentes, es fundamental comprender cómo almacenar, acceder y gestionar estos datos de manera eficiente. Las bases de datos documentales, como MongoDB, ofrecen una solución poderosa y flexible para manejar grandes volúmenes de datos estructurados y semi-estructurados. Esta guía proporciona una introducción a las bases de datos documentales y a MongoDB, orientada a facilitar el aprendizaje y la comprensión de estos conceptos esenciales.

## Bases de Datos Documentales

Una base de datos documental es un tipo de base de datos NoSQL diseñada para almacenar, recuperar y gestionar información orientada a documentos. A diferencia de las bases de datos relacionales, que almacenan datos en tablas con filas y columnas, las bases de datos documentales utilizan documentos (generalmente en formatos JSON, BSON, etc.) para almacenar datos de manera más flexible. Esta estructura permite representar relaciones complejas y estructuras de datos anidadas de manera intuitiva y eficiente.

En el contexto de las bases de datos documentales, un "documento" se refiere a una unidad de almacenamiento de datos, que generalmente encapsula información relacionada en un formato estructurado. Este puede contener tipos de datos variados, como texto, números, booleanos, arrays, y objetos anidados, representando de manera integral los datos relacionados a un objeto o entidad.

Los documentos son análogos a los registros o filas en las bases de datos relacionales, pero ofrecen una flexibilidad mucho mayor en términos de la estructura de datos que pueden contener.

### Ejemplos de documentos:

Un documento, como elemento para almacenar información, puede ser entendido como una estructura de datos compuesta por elementos clave-valor que almacena información relevante de manera organizada. A lo largo de los años, han surgido diversos formatos para facilitar la representación, almacenamiento y transferencia de estos documentos, cada uno con sus propias características y ventajas.

De entre las diversas opciones disponibles, dos formatos se destacan por su amplia adopción y versatilidad: JSON (JavaScript Object Notation) y XML (eXtensible Markup Language). Ambos formatos ofrecen una solución eficiente para el manejo de datos en aplicaciones modernas, permitiendo una fácil integración, interoperabilidad y manipulación de datos entre diferentes sistemas.

- JSON se ha consolidado como uno de los formatos más populares debido a su simplicidad, eficiencia y fácil legibilidad tanto para máquinas como para humanos. Su estructura ligera y el uso de texto lo hacen ideal para el intercambio de datos en la web y el almacenamiento en bases de datos documentales.
- XML, con su capacidad para representar datos complejos y su flexibilidad en la definición de esquemas, proporciona un mecanismo robusto para estructurar

documentos de manera jerárquica, permitiendo una amplia gama de aplicaciones, desde configuraciones de software y servicios web hasta intercambios de datos en entornos empresariales.

A continuación, se presentan un ejemplo de cómo representar la información de contacto utilizando tanto un documento en formato JSON como un documento en formato XML, mostrando la flexibilidad y capacidad de cada formato para estructurar datos complejos de manera eficiente y clara.



```json
{
    "nombre": "Juan Pérez",
    "email": "juan.perez@example.com",
    "telefonos": [
      {
        "tipo": "movil",
        "numero": "+34 600 000 000"
      },
      {
        "tipo": "casa",
        "numero": "+34 910 000 000"
      },
      {
        "tipo": "trabajo",
        "numero": "+34 910 123 456"
      }
    ],
    "direcciones": [
      {
        "calle": "Av. Libertad 123",
        "ciudad": "Madrid",
        "pais": "España"
      },
      {
        "calle": "Calle Sol 456",
        "ciudad": "Barcelona",
        "pais": "España"
      }
    ]
  }
```

Documento JSON con datos de contacto



```xml
<contacto>
  <nombre>Juan Pérez</nombre>
  <email>juan.perez@example.com</email>
  <telefonos>
    <telefono tipo="movil">+34 600 000 000</telefono>
    <telefono tipo="casa">+34 910 000 000</telefono>
    <telefono tipo="trabajo">+34 910 123 456</telefono>
  </telefonos>
  <direcciones>
    <direccion>
      <calle>Av. Libertad 123</calle>
      <ciudad>Madrid</ciudad>
      <pais>España</pais>
    </direccion>
    <direccion>
      <calle>Calle Sol 456</calle>
      <ciudad>Barcelona</ciudad>
      <pais>España</pais>
    </direccion>
  </direcciones>
</contacto>
```

Documento XML con datos de contacto

En ambos ejemplos se muestra la representación del mismo contacto personal con varios números de teléfono y varias direcciones, utilizando una estructura clara y legible, representando datos jerárquicos complejos mediante el uso de etiquetas y atributos.

En el ejemplo JSON se puede ver que utiliza una sintaxis basada en objetos **{ }** y arrays **[ ]** para organizar los datos. Los objetos representan conjuntos de pares *clave-valor*, donde cada *clave* es una cadena de texto entre comillas dobles **"**, y el *valor* puede ser una cadena, un número, una fecha, un objeto, un array, valor **true**, valor **false**, valor **null**, etc.

Sin embargo XML utiliza *etiquetas personalizadas* para delimitar los elementos y puede incluir atributos dentro de estas etiquetas para proporcionar información adicional. Los elementos pueden, a su vez, contener otros elementos. Cada elemento en XML se marca con una etiqueta de inicio, como **<nombre>**, y una etiqueta de cierre, como **</nombre>**, para delimitar el contenido.

Para continuar con este ejemplo, se va a ilustrar la flexibilidad que proporcionan las bases de datos documentales. Si se quiere hacer una modificación en el ejemplo anterior añadiendo también información de las redes sociales de la persona, los cambios

empleando una base de datos documental serían sencillos y simplemente implicaría incorporarla directamente en los documentos. Por ejemplo, al contacto anterior se le van a añadir dos redes sociales, X (antes Twitter) y LinkedIn, y se haría como se muestra a continuación.



```
      "ciudad": "Barcelona",
      "pais": "España"
    }
  ],
  "redesSociales": {
    "Twitter": "@juanperez",
    "LinkedIn": "linkedin.com/in/juanperez"
  }
}
```

Redes sociales añadidas al documento JSON



```
      <ciudad>Barcelona</ciudad>
      <pais>España</pais>
    </direccion>
  </direcciones>
  <redesSociales>
    <redSocial tipo="Twitter">@juanperez</redSocial>
    <redSocial tipo="LinkedIn">linkedin.com/in/juanperez</redSocial>
  </redesSociales>
</contacto>
```

Redes sociales añadidas al documento XML

En una base de datos relacional, para modelar el mismo ejemplo que permita almacenar la misma información de contacto, incluyendo varios números de teléfono y varias direcciones, se necesitarían, al menos, las siguientes tres tablas para representar la información del contacto de manera normalizada:

1. Contactos: Contendría la información básica del contacto.
2. Telefonos: Almacenaría los números de teléfono de cada contacto, permitiendo múltiples entradas por contacto.
3. Direcciones: Guardaría las direcciones asociadas a cada contacto, también permitiendo múltiples entradas por contacto.

Y para los datos del contacto visto antes, en cada una de las tablas debería de haber un contenido como el siguiente:

**Contactos**

| ContactoID | Nombre | Email |
|---|---|---|
| 1 | Juan Pérez | juan.perez@example.com |

**Telefonos**

| TelefonoID | ContactoID | Tipo | Numero |
|---|---|---|---|
| 1 | 1 | movil | +34 600 000 000 |
| 2 | 1 | casa | +34 910 000 000 |
| 3 | 1 | trabajo | +34 910 123 456 |

**Direcciones**

| DireccionID | ContactoID | Calle | Ciudad | Pais |
|---|---|---|---|---|
| 1 | 1 | Av. Libertad 123 | Madrid | España |
| 2 | 1 | Calle Sol 456 | Barcelona | España |

Estructura y contenido de base de datos relacional para almacenar el contacto de ejemplo

Si ahora se quiere hacer una modificación añadiendo también información de las redes sociales del contacto, los cambios serían de mucho calado implicando incluso cambiar el esquema de la base de datos creando una nueva tabla *RedesSociales* que almacenaría las cuentas de redes sociales asociadas a cada contacto. Para el mismo ejemplo, debería contener algo similar a:

**RedesSociales**

| RedSocialID | ContactoID | Tipo | Usuario |
|---|---|---|---|
| 1 | 1 | Twitter | @juanperez |
| 2 | 1 | LinkedIn | linkedin.com/in/juanperez |

Contenido tabla RedesSociales necesaria

La importancia de los cambios realizados en la base de datos relacional, únicamente para poder incorporar la información de las redes sociales, se puede también visualizar en el diagrama Entidad-Relación (E-R) antes y después de la modificación, como muestran las dos imágenes siguientes:

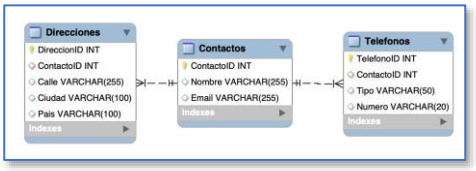



Diagrama E-R de contactos

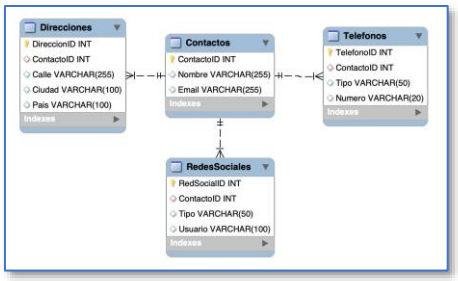



Diagrama E-R de contactos con redes sociales

# MongoDB

## Introducción

MongoDB emergió como una solución innovadora en el mundo de las bases de datos en el año 2007, fruto del trabajo de la compañía 10gen, que actualmente se conoce como MongoDB Inc. Su diseño respondía a la necesidad de abordar los desafíos de escalabilidad y agilidad que enfrentaban las aplicaciones web modernas.

En un movimiento estratégico hacia la colaboración y la innovación abierta, MongoDB fue lanzado como OpenSource en 2009 bajo la licencia AGPL, una decisión que ha facilitado su adopción y evolución gracias a las contribuciones de una comunidad global de desarrolladores.

El nombre de MongoDB proviene de la palabra "*humongous*", reflejando su capacidad para manejar voluminosos conjuntos de datos de manera eficiente. Este origen etimológico es una promesa de la capacidad de MongoDB para escalar junto con las necesidades de almacenamiento de datos de las aplicaciones que crecen a un ritmo sin precedentes.

MongoDB se clasifica como una **base de datos orientada a documentos**. Esto significa que, a diferencia de las bases de datos relacionales tradicionales que almacenan datos en registros dispuestos en tablas, MongoDB almacena la información en documentos. Estos documentos, generalmente en formato BSON (una versión binaria de JSON), permiten una representación más rica y flexible de los datos.

Una diferencia fundamental entre MongoDB y las bases de datos relacionales es su naturaleza **'schemaless'**, o sin esquema fijo. Esto significa que los documentos dentro de una misma colección —un concepto que puede considerarse similar a las tablas en el mundo relacional— pueden tener esquemas distintos entre sí. Tal flexibilidad permite una mayor libertad en la gestión de los datos y reduce la complejidad en el manejo de los cambios de esquemas que son comunes durante el ciclo de vida del desarrollo de aplicaciones.

Este enfoque 'schemaless' es particularmente ventajoso en el desarrollo ágil y en entornos que requieren una rápida iteración y donde los modelos de datos pueden cambiar a lo largo del tiempo. Facilita la adaptación a los cambios y la incorporación de nuevos tipos de datos sin la necesidad de realizar modificaciones estructurales o migraciones de esquemas costosas y complejas.

MongoDB se ha ganado su lugar como una herramienta esencial en el arsenal de desarrolladores modernos, ingenieros de datos y arquitectos de sistemas gracias a su diseño intuitivo, flexibilidad y rendimiento robusto. Su enfoque orientado a documentos, junto con su modelo operativo 'schemaless', ofrece una solución eficaz para la gestión de datos en la era digital, donde la agilidad y la capacidad de adaptarse rápidamente a nuevas necesidades de datos son clave.

## Aplicaciones de MongoDB

MongoDB, con su naturaleza flexible y escalable, es una elección predilecta para una diversidad de aplicaciones que requieren gestión eficiente de grandes volúmenes de

datos, velocidad de acceso y complejas estructuras de datos. Algunas de las aplicaciones más destacadas de MongoDB incluyen:

### Internet de las cosas (Internet of Things, IoT)

En el ámbito del Internet de las Cosas, MongoDB se utiliza para recoger y procesar los datos generados por una multitud de dispositivos conectados. Su capacidad para manejar grandes flujos de datos y estructuras variadas la hace ideal para almacenar lecturas de sensores, automatizar acciones en respuesta a eventos y realizar análisis en tiempo real, permitiendo así una gestión más inteligente y eficiente del ecosistema IoT.

### Visualización Geoespacial

MongoDB sobresale en la gestión de datos geoespaciales, proporcionando tipos de datos y operadores específicos para la consulta de información basada en la localización. Esto lo convierte en una herramienta poderosa para aplicaciones que requieren la visualización geoespacial de datos, como servicios de mapeo, seguimiento de activos y aplicaciones de planificación urbana.

### Gestión de Contenidos

Plataformas como Sourceforge se benefician de las capacidades de MongoDB para la gestión de contenidos. Al poder almacenar documentos y metadatos con esquemas flexibles, MongoDB se adapta a diferentes tipos de contenido y su evolución a lo largo del tiempo, lo cual es esencial para plataformas que administran una gran variedad de proyectos de software y sus respectivas comunidades.

### Aplicaciones Móviles

Empresas como Expedia utilizan MongoDB para impulsar sus aplicaciones móviles, donde es fundamental contar con una base de datos que pueda servir contenido personalizado y realizar reservas en tiempo real a escala global, manejando picos de tráfico y una gran variedad de consultas de usuario.

### Videojuegos

En el desarrollo de videojuegos, como FIFA Online, MongoDB juega un papel crítico al gestionar la información de estado del juego, las interacciones de los jugadores y las estadísticas en vivo. Su rendimiento y escalabilidad hacen posible una experiencia de juego en línea fluida y reactiva, incluso durante los momentos de alta demanda.

### Log de Eventos

Plataformas con enormes cantidades de usuarios, como Facebook, generan una cantidad significativa de datos de eventos. MongoDB puede manejar eficientemente estos logs, permitiendo el análisis y la monitorización de la actividad en la aplicación para mejorar la experiencia del usuario y proporcionar un servicio más confiable.

### Otros Casos de Éxito

MongoDB también es utilizado por empresas líderes en diversos sectores, incluyendo eBay en el comercio electrónico, Orange en telecomunicaciones, Barclays en servicios financieros, Adobe en software creativo y Telefónica en servicios integrados de

comunicaciones. Estas organizaciones confían en MongoDB por su alta disponibilidad, su soporte para un procesamiento complejo de datos y su facilidad de uso para desarrolladores.

Las aplicaciones de MongoDB se están expandiendo constantemente, demostrando su valor como una base de datos que no solo puede manejar las necesidades actuales de almacenamiento de datos y análisis, sino que también está preparada para el futuro, adaptable a las nuevas tendencias y requisitos del mercado en constante evolución.

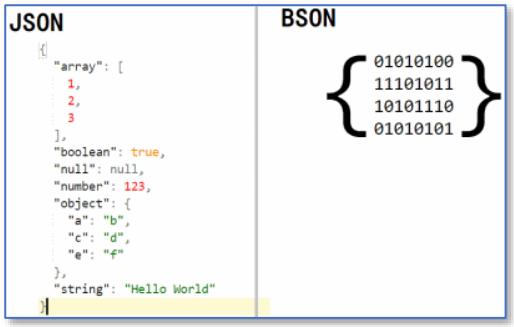
## Modelo de datos de MongoDB

El modelo de datos de MongoDB es una de las características distintivas que lo definen como una **base de datos NoSQL orientada a documentos**. En MongoDB, los datos se almacenan en documentos, que son entidades autocontenidas y con capacidad de autodescribirse. Estos documentos se agrupan en colecciones, que son el equivalente funcional a las tablas en las bases de datos relacionales, pero con una flexibilidad mucho mayor.

### Documentos en Formato BSON

Los documentos en MongoDB no se almacenan en el formato JSON estándar, sino en una representación binaria llamada BSON (Binary JSON). BSON extiende el modelo JSON en varias formas importantes:

- **Tipos de Datos Adicionales**: Mientras que JSON está limitado a tipos de datos como texto, número, array, booleano, y null, BSON introduce tipos adicionales más ricos y útiles para el desarrollo, como Date (fechas), BinData (datos binarios), y ObjectId (identificadores únicos), entre otros.
- **Más Eficiencia en Espacio**: Aunque BSON requiere más espacio que JSON debido a su codificación binaria, este formato está optimizado para ser más eficiente en espacio en comparación con una representación de JSON equivalente almacenada en UTF-8. Esto se debe a que BSON puede representar ciertos tipos de datos, como números y fechas, de manera más compacta.
- **Diseñado para Búsquedas más Eficientes**: BSON está diseñado teniendo en cuenta la eficiencia de las operaciones de lectura/escritura y la capacidad de indexación. Al ser un formato binario, facilita la serialización y deserialización rápida de los documentos, lo que resulta en un acceso más rápido a los datos. Los índices en MongoDB, que pueden ser creados en cualquier campo de un documento, mejoran significativamente el rendimiento de las búsquedas, haciendo las consultas más eficientes y rápidas.



La elección de BSON como formato de representación de documentos en MongoDB subraya el compromiso de la base de datos con el rendimiento, la eficiencia y la flexibilidad. La naturaleza binaria de BSON también significa que puede ser analizada directamente en la mayoría de los lenguajes de programación sin la necesidad de convertir los datos a una cadena de texto, como se haría con JSON, lo cual es una ventaja significativa en términos de rendimiento.

El modelo de datos de MongoDB y su uso de BSON permiten a los desarrolladores trabajar con un sistema de base de datos que no solo es eficiente y rápido, sino también intuitivo y fácil de usar, reflejando la estructura y los tipos de datos utilizados en el código de la aplicación. Esto reduce la cantidad de lógica de traducción necesaria entre la aplicación y la base de datos, lo que a su vez permite un desarrollo más rápido y directo.

## Tipos de datos en MongoDB

MongoDB ofrece una amplia gama de tipos de datos que son representados en el formato BSON, permitiendo almacenar datos con una gran diversidad de tipos y características. A continuación, se detallan algunos de los tipos de datos más comunes que se pueden encontrar en MongoDB:

### Numéricos

- Double: Este es el tipo de dato por defecto para números en MongoDB. Utiliza 64 bits para almacenar un número en formato de coma flotante, lo que proporciona una precisión adecuada para la mayoría de los cálculos numéricos en las aplicaciones.
- Decimal: Especialmente diseñado para cálculos de precisión decimal, el tipo Decimal o Decimal128 utiliza 128 bits, lo que permite una alta precisión en operaciones financieras y científicas donde se requiere que los cálculos sean extremadamente precisos.
- Int: Representa valores enteros de hasta 32 bits. Es útil para números que no requieren decimales y donde se sabe que el valor no excederá el rango de un entero de 32 bits.
- Long: Para valores enteros más grandes, MongoDB utiliza el tipo Long, que emplea 64 bits y es capaz de almacenar números en un rango mucho mayor que Int.

### Texto

- String: MongoDB almacena cadenas de texto como String en formato UTF-8, lo que le permite manejar texto en una amplia variedad de idiomas y caracteres especiales.
- Regex: El tipo Regex se usa para almacenar expresiones regulares, que pueden ser utilizadas luego en búsquedas y validaciones dentro de la base de datos. Esto es particularmente útil para patrones de búsqueda complejos.

### Fecha

- **Date**: En MongoDB, el tipo Date es un entero de 64 bits que representa la cantidad de milisegundos desde la medianoche del 1 de enero de 1970 UTC. Este formato es compatible con el estándar de tiempo Unix y es adecuado para representar fechas y horas.
- **Timestamp**: El Timestamp es un valor especial en MongoDB que se utiliza internamente en el sistema de replicación. Es un entero de 64 bits donde los primeros 32 bits son un valor de tiempo Unix (segundos desde la época Unix, que es similar al tipo Date) y los últimos 32 bits son un ordinal incremental que se usa para ordenar las operaciones en un único segundo. Aunque puede usarse para marcas de tiempo, su principal función es en el control de la concurrencia y el orden de las operaciones.

Cada uno de estos tipos de datos está diseñado para satisfacer diferentes necesidades y casos de uso dentro de las aplicaciones que utilizan MongoDB, desde el almacenamiento de datos simples hasta el manejo de complejas estructuras de datos y operaciones precisas. La elección del tipo de dato correcto es crucial para el rendimiento y la exactitud de las operaciones en la base de datos.

## Ejemplo:

Vamos a crear un documento MongoDB donde se utilizan diferentes tipos de datos para almacenar información variada sobre una persona:

- **ObjectId** para el campo ***_id***, proporcionando un identificador único generado por MongoDB.
- **String** para el *nombre*, *dirección* y números de teléfono, almacenando texto.
- **Int** para la *edad*, ya que es un valor entero.
- **Double** para el *peso*, que puede necesitar valores decimales.
- **Decimal** para *saldo_bancario*, donde se requiere precisión para representar cantidades monetarias.
- **Date** para *fecha_nacimiento*, almacenando una fecha específica.
- **Timestamp** para *ultima_sesion*, que podría usarse para versiones o para registrar el momento exacto de eventos en la base de datos con un ordinal para operaciones que ocurrieron en el mismo segundo.
- **Regex** para *patrones_busqueda*, permitiendo búsquedas avanzadas basadas en patrones dentro de la base de datos.

El documento se vería como se muestra a continuación:

```
{
  "_id": ObjectId("507f191e810c19729de860ea"), // Tipo especial ObjectId para identificadores únicos
  "nombre": "Juan Pérez", // Tipo String para texto
  "edad": 30, // Tipo Int para números enteros
  "peso": 70.5, // Tipo Double para números con coma flotante
  "saldo_bancario": Decimal("12345.67"), // Tipo Decimal para representar valores monetarios con alta precisión
  "fecha_nacimiento": ISODate("1990-05-15T00:00:00Z"), // Tipo Date para fechas
  "ultima_sesion": Timestamp(1615005725, 1), // Tipo Timestamp para marcas de tiempo con ordinales
  "direccion": {
    "calle": "Av. Libertad 123",
    "ciudad": "Madrid",
    "codigo_postal": "28080"
  },
  "patrones_busqueda": /mongo.*/i, // Tipo Regex para expresiones regulares
  "telefono": {
    "movil": "+34 600 000 000", // Tipo String para texto
    "casa": "+34 910 000 000" // Tipo String para texto
  }
}
```

Documento MongoDB con datos de diversos tipos

## Más tipos de datos en MongoDB

Además de los tipos de datos numéricos y de texto básicos, MongoDB soporta una variedad de tipos de datos especiales que permiten una gran flexibilidad en la manipulación y almacenamiento de datos.

### Array

El tipo de dato Array en MongoDB es una estructura de datos particularmente versátil, ya que puede contener una lista de elementos que pueden ser de cualquier tipo de datos BSON, incluidos otros arrays, lo que posibilita la creación de estructuras de datos anidadas y complejas. Un array se define utilizando corchetes **[]**, y sus elementos están separados por comas. Esta sintaxis es muy similar a la que se usa en JSON y en muchos lenguajes de programación, lo que hace que su uso sea intuitivo para los desarrolladores.

Por ejemplo, si quisiéramos almacenar una lista de números de teléfono para un contacto en MongoDB, podríamos tener un documento que incluya un campo **telefonos** definido como un array de esta manera:

```
{
  "nombre": "Juan Pérez",
  "telefonos": ["+34 600 000 000", "+34 910 000 000", "+34 910 123 456"]
}
```

En este caso, *telefonos* es un array que contiene tres elementos, cada uno de los cuales es un String que representa un número de teléfono diferente.

Los arrays no están limitados a almacenar solo un tipo de dato; pueden contener una combinación de tipos, aunque generalmente es recomendable mantenerlos homogéneos para evitar complejidades innecesarias en la lógica de la aplicación. Por ejemplo, un array puede contener *strings*, números, objetos u otros arrays, todos ellos juntos:

```
{
  "datosMixtos": ["texto", 123, {"clave": "valor"}, [1, 2, 3]]
}
```

### ObjectId

Es un tipo de dato especial utilizado principalmente para el campo ***_id,*** que es un identificador único para cada documento en una colección. El *ObjectId* está diseñado para ser rápido en la generación de valores únicos a través de máquinas, ser compacto y ser ordenable por el tiempo de creación del documento.

```
{
    "_id": ObjectId("507f191e810c19729de860ea")
}
```

### Boolean

Los valores booleanos en MongoDB son un tipo de dato que representa un *verdadero* o *falso* (***true*** o ***false***) y es fundamental para la lógica de control y las condiciones dentro de las consultas y el procesamiento de datos.

```
{
  "esClientePremium": true
}
```

### Null

El valor ***Null*** en MongoDB representa la ausencia de un valor y se utiliza para denotar que un campo está intencionalmente vacío, indefinido o con valor desconocido.

```
{
    "direccionSecundaria": null
}
```

### Javascript

MongoDB tiene la capacidad única de almacenar fragmentos de código Javascript directamente en los documentos. Esto puede ser útil para funciones que se ejecutan dentro de la base de datos, como en las consultas o agregaciones, que se verán más adelante en este documento.

```
{
    "calcularDescuento": function(precio) {
      if (precio > 100) {
        return precio * 0.8; // 20% de descuento
      } else {
        return precio * 0.9; // 10% de descuento
      }
    }
}
```

Cada uno de estos tipos de datos especiales tiene un propósito y uso específico dentro de MongoDB, ofreciendo a los desarrolladores opciones flexibles para modelar sus datos de manera que se alineen mejor con la lógica de la aplicación y las operaciones de la base de datos.

## Conceptos Básicos del Modelo de Datos de MongoDB

MongoDB introduce un modelo de datos flexible y dinámico que permite a los desarrolladores almacenar y manipular datos de una manera más natural y potente en

comparación con las bases de datos relacionales tradicionales. A continuación, se describen los elementos fundamentales de este modelo:

### Documento (document)

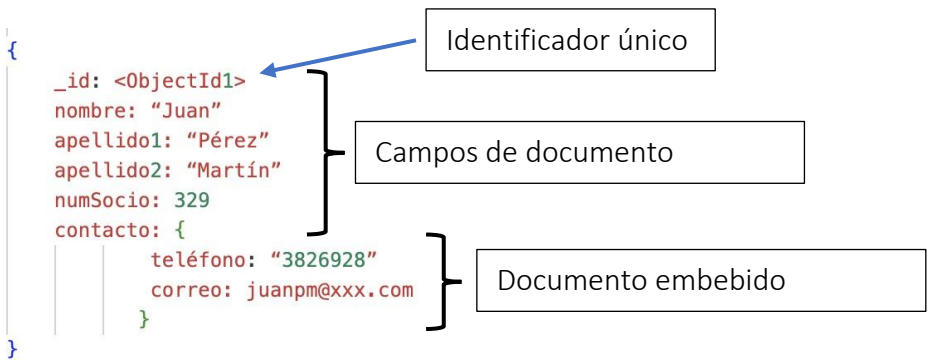
Un documento en MongoDB es un **conjunto de pares campo-valor** y representa la unidad básica de almacenamiento, similar a lo que sería una "*fila*" en una base de datos relacional. Los documentos en MongoDB se caracterizan por:

- **Sin esquema predefinido**: No hay una estructura fija o esquema que los documentos deban seguir, lo que significa que dentro de la misma colección, cada documento puede tener un conjunto diferente de campos.
- **Nombre del campo junto a su valor**: Cada campo dentro de un documento tiene un nombre único seguido de su valor, que puede ser de cualquier tipo de datos BSON.
- **No hay valores nulos**: Aunque MongoDB soporta el tipo de datos *null*, no se almacenan campos nulos; si un campo está presente, tiene un valor asignado, aunque ese valor pueda ser *null*.
- **Puede contener otros documentos**: Los documentos pueden contener otros documentos embebidos, permitiendo representar estructuras de datos complejas y jerarquías.
- **Campo _id siempre presente**: Cada documento debe tener un campo _id, que asegura la unicidad dentro de la colección. Si no se proporciona, MongoDB lo generará automáticamente.

### Colección (collection)

Una colección en MongoDB es un **conjunto de documentos** y es comparable a una "*tabla*" en las bases de datos relacionales. Las características de las colecciones incluyen:

- **Sin esquema**: Similar a los documentos, las colecciones no requieren un esquema fijo. Los documentos dentro de una colección pueden tener diferentes campos.
- **Identificador único**: Cada documento en una colección tiene un campo ***_id*** que actúa como un identificador único. Si al insertar un documento no se especifica este campo, MongoDB generará automáticamente un valor *ObjectId*.



### Campo (*field*)

Los campos en los documentos de MongoDB son análogos a las "*columnas*" en las bases de datos relacionales y tienen las siguientes propiedades:

- Valor asignado: Cada campo en un documento tiene un valor asociado, que puede ser de cualquier tipo de datos BSON.
- Índices: Se pueden crear índices sobre los campos para optimizar las consultas y mejorar el rendimiento de la base de datos. Los índices permiten una recuperación más rápida de los datos.
- Campo _id siempre presente: Cada documento debe tener un campo _id, que asegura la unicidad dentro de la colección. Si no se proporciona, MongoDB lo generará automáticamente.

Estos conceptos básicos reflejan la filosofía de diseño de MongoDB, orientada a maximizar la flexibilidad y la eficiencia en el manejo de datos. Esta flexibilidad es particularmente valiosa en aplicaciones que requieren un desarrollo rápido y que pueden evolucionar con el tiempo, ya que permite a los desarrolladores adaptar la estructura de los datos a medida que cambian los requisitos de la aplicación sin necesidad de realizar migraciones de esquema costosas y complejas.

## Ejemplo de colección de documentos

Una colección en MongoDB es una agrupación de documentos que a menudo se utiliza para almacenar registros relacionados, similar a cómo una tabla agrupa filas en una base de datos relacional. A continuación, se muestra un ejemplo de cómo podría verse una colección llamada **`usuarios`**, que incluye documentos con varios campos, incluyendo algunos que contienen otros documentos.

```
// COLLECIÓN DE "SOCIOS"
[
    { // DOCUMENTO 1
      "_id": ObjectId("507f1f77bcf86cd799439011"),
      "nombre": "Ana Ramos",
      "edad": 25,
      "email": "ana.ramos@example.com",
      "direcciones": [
        {"tipo": "principal", "calle": "Av. de la Constitución 123", "ciudad": "Madrid", "CP": "28080"},
        {"tipo": "vacacional", "calle": "Calle del Sol 15", "ciudad": "Málaga", "CP": "29015"}
      ],
      "aficiones": ["lectura", "pintura", "escalada"]
    },

    { // DOCUMENTO 2
      "_id": ObjectId("507f1f77bcf86cd799439012"),
      "nombre": "Luis Molina",
      "edad": 30,
      "email": "luis.molina@example.com",
      "direcciones": [
        {"tipo": "principal", "calle": "Paseo de Gracia 45", "ciudad": "Barcelona", "CP": "08007"}
      ],
      "aficiones": ["fútbol", "viajar"]
    },

    {   // DOCUMENTO 3
      "_id": ObjectId("507f1f77bcf86cd799439013"),
      "nombre": "Sofía Núñez",
      "edad": 40,
      "email": "sofia.nunez@example.com",
      // Este documento no tiene campo "direcciones" ni "aficiones" -> Gran flexibilidad
    }
  ]
```

En este ejemplo, la colección **`usuarios`** contiene tres documentos, cada uno de los cuales representa un usuario. Observemos varios puntos clave en relación con los conceptos básicos de MongoDB:

- Documento: Cada elemento de la colección (entre llaves {}) es un documento con pares campo-valor. Por ejemplo, **`"nombre": "Ana Ramos"`** es un campo con su correspondiente valor.
- Sin esquema predefinido: Cada documento puede tener diferentes campos. Mientras Ana Ramos tiene campos para **`direcciones`** y **`aficiones`**, Sofía Núñez no los tiene.
- Jerarquía de documentos: Dentro de los documentos, existen campos como **`direcciones`**, que contienen arrays de otros documentos, demostrando la capacidad de crear jerarquías de datos.
- Colección: La colección **`usuarios`** agrupa todos los documentos, que son independientes entre sí en cuanto a su estructura.
- Identificador único (_id): Cada documento tiene un campo **`_id`** que es generado automáticamente por MongoDB si no se especifica uno. Este campo sirve como identificador único dentro de la colección. MongoDB automáticamente crea un índice en el campo _id para cada colección, asegurando que cada documento tenga un identificador único y que las operaciones de búsqueda por _id sean eficientes.
- Campos: Los documentos contienen campos como **`nombre`**, **`edad`**, y **`email`**, etc. cada uno con valores asignados, y son similares a las columnas en las bases de datos relacionales.

Este ejemplo muestra la naturaleza flexible y dinámica de las colecciones y documentos en MongoDB, lo que facilita la adaptación a las necesidades de datos cambiantes y complejas de las aplicaciones modernas.

## Factores Importantes en el Modelo de Datos de MongoDB

El modelo de datos en MongoDB se aparta significativamente de las metodologías tradicionales empleadas en bases de datos relacionales. Aquí se destacan algunas características importantes que definen su enfoque:

### Metodología no Establecida

MongoDB es flexible por naturaleza y, a diferencia de las bases de datos relacionales, no se basa en una metodología de diseño de datos claramente establecida. No existe una "forma correcta" de modelar sus datos. La estructura del modelo se adapta a las necesidades de la aplicación y se centra en el rendimiento y la eficiencia de las consultas.

### Entorno “más rico” que SQL

MongoDB proporciona un conjunto de características que ofrecen más flexibilidad en comparación con las bases de datos basadas en SQL:

- Tablas embebidas: En MongoDB, es común embeber documentos dentro de otros. Por ejemplo, en lugar de tener una tabla separada para direcciones, puedes

tener un array de direcciones dentro de cada documento con información del usuario.

- Documentos embebidos con nivel de profundidad arbitrario: Puedes embeber documentos dentro de otro documento, creando estructuras de datos jerárquicas con varios niveles de profundidad, lo que sería equivalente a múltiples relaciones de tablas en SQL.
- Claves que pueden faltar: No todos los documentos necesitan tener el mismo conjunto de claves (campos), lo que aporta gran flexibilidad en la representación de datos.
- Documentos con esquemas distintos en la misma colección: Es posible, y a veces deseable, tener documentos con diferentes esquemas dentro de la misma colección.

### Consideraciones al Diseñar con MongoDB

Al modelar datos para MongoDB, hay prácticas comunes de bases de datos relacionales que debemos reconsiderar:

- JOINS: MongoDB no realiza operaciones de JOIN como las bases de datos relacionales. En su lugar, se embeben los documentos relacionados dentro de un documento principal o se realizan múltiples consultas y se ensamblan los datos en la aplicación.
- Foreign keys: No hay restricciones de claves foráneas en MongoDB; la integridad referencial debe ser mantenida por la lógica de la aplicación si es necesaria.
- Transacciones: Tradicionalmente, MongoDB se ha centrado en operaciones atómicas a nivel de documento individual. Aunque las versiones recientes han introducido transacciones multi-documento en MongoDB, estas pueden no tener el mismo nivel de madurez o rendimiento que las transacciones en sistemas SQL y se deben usar con precaución.

El modelo de datos de MongoDB está orientado al rendimiento y a la representación eficiente de datos complejos y jerárquicos. Permite a los desarrolladores un desarrollo más rápido y una mayor flexibilidad para evolucionar el esquema de datos con el tiempo, pero también exige una comprensión clara de sus operaciones y limitaciones para diseñar una solución eficaz.

## Ejemplo de base de datos: solución relacional y soluciones MongoDB

En este ejemplo se va a diseñar una base de datos para almacenar la información de los libros de una librería, tanto de los autores como de los títulos de los libros. En el diseño hay que contemplar la posibilidad de que un autor puede escribir varios libros, pero cada libro tiene un solo autor.

### Solución con base de datos relacional

Empleando el modelo relacional, para implementar esta base de datos se crearían dos tablas: Autores y Libros con los siguientes atributos:

- **Autores**( AutorID, Nombre, Nacionalidad)
- **Libros** (LibroID, Título, Año, AutorID)

Por ejemplo, el contenido de estas tablas para almacenar dos libros de García Márquez y un libro de Isabel Allende, podría ser el mostrado a continuación:

Tabla `Autores`

| AutorID | Nombre | Nacionalidad |
|---|---|---|
| 1 | Gabriel García Márquez | Colombiana |
| 2 | Isabel Allende | Chilena |

Tabla `Libros`

| LibroID | Título | Año | AutorID |
|---|---|---|---|
| 1 | Cien años de soledad | 1967 | 1 |
| 2 | El amor en los tiempos del cólera | 1985 | 1 |
| 3 | La casa de los espíritus | 1982 | 2 |

Para obtener una lista de libros junto con la información de sus autores, habría que realizar una consulta SQL con un JOIN:

```
SELECT Libros.Título, Libros.Año, Autores.Nombre, Autores.Nacionalidad
FROM Libros JOIN Autores ON Libros.AutorID = Autores.AutorID;
```

Este JOIN combina las filas de las dos tablas basándose en la condición de que el ***AutorID*** de **Libros** coincida con el ***AutorID*** de **Autores**.

## Soluciones con MongoDB

Al diseñar una base de datos para la librería en MongoDB, hay dos posibles aproximaciones:

1. Crear una **colección de libros**
2. Crear una **colección de autores**

Ambas soluciones son válidas, y la elección entre una y otra depende en gran medida de los patrones de acceso a los datos y de las operaciones más comunes que se realizarán en el futuro en la base de datos.

### Solución: crear colección de libros

Esta aproximación coloca los libros en el centro del modelo de datos. Es eficiente si las operaciones más comunes implican buscar libros por título, año o si el acceso directo a la información del libro es prioritario. Si se actualizan con frecuencia detalles específicos del libro o si se buscan libros sin necesidad de detalles del autor, este modelo puede ser el más adecuado. Además, facilitaría en el futuro la agregación de datos relacionados con los libros, como ventas o reseñas.

Una colección de libros podría llevar a redundancia en la información del autor si varios libros comparten el mismo autor.

Para el ejemplo, la colección **`libros`** podría tener los siguientes documentos, que incluyen la información del autor directamente embebida:

```
[{  "_id": ObjectId("507f1f77bcf86cd799439011"),
      "título": "Cien años de soledad",
      "año": 1967,
      "autor": {  "nombre": "Gabriel García Márquez",
                  "nacionalidad": "Colombiana"    }
   },
   {   "_id": ObjectId("507f1f77bcf86cd799439012"),
      "título": "El amor en los tiempos del cólera",
      "año": 1985,
      "autor": {    "nombre": "Gabriel García Márquez",
                  "nacionalidad": "Colombiana"    }
   },
   {   "_id": ObjectId("507f1f77bcf86cd799439013"),
      "título": "La casa de los espíritus",
      "año": 1982,
      "autor": {  "nombre": "Isabel Allende",
                  "nacionalidad": "Chilena"   }
   } ]
```

En esta estructura, cada documento de libro contiene toda la información necesaria, y no es necesario realizar un JOIN para obtener los datos del autor cuando se consulta un libro. Para obtener todos los libros junto con la información del autor, simplemente realizaríamos una consulta a la colección libros:

```
db.libros.find({})
```

Esta consulta devuelve todos los documentos en la colección libros, cada uno incluyendo la información del autor. En MongoDB, la consulta es más directa porque los datos están desnormalizados y embebidos dentro del mismo documento, lo que elimina la necesidad de operaciones de JOIN. Esto puede aumentar el rendimiento para operaciones de lectura, pero también significa que cualquier actualización de la información del autor requeriría actualizar todos los documentos donde está embebida la información de ese autor.

### Solución: crear colección de autores

En este caso, los autores son el núcleo de la base de datos, y cada autor tiene un array de libros que ha escrito. Esta estructura es útil si las operaciones más comunes son en torno a los autores y si se desea obtener fácilmente toda su información. Si las actualizaciones a la información del autor son frecuentes o si se desea analizar la obra de un autor en su conjunto, este enfoque es el adecuado.

Una colección de autores podría llevar a una actualización más sencilla de los datos del autor pero dificulta la gestión individual de los libros.

Para el ejemplo, la colección **`autores`** podría tener los siguientes documentos, que incluyen la información del autor directamente embebida:

```
[   { "_id": ObjectId("507f1f77bcf86cd799439011"),
     "nombre": "Gabriel García Márquez",
     "nacionalidad": "Colombiana",
     "libros": [
           {"título": "Cien años de soledad", "año": 1967},
```

```
        {"título": "El amor en los tiempos del cólera", "año": 1985 }]
    },
    { "_id": ObjectId("507f1f77bcf86cd799439012"),
      "nombre": "Isabel Allende",
      "nacionalidad": "Chilena",
      "libros": [
        {"título": "La casa de los espíritus", "año": 1982 }]
     }
]
```

En este modelo, cada autor es un documento que contiene un campo **`libros`**, que es un array de documentos representando cada libro escrito por el autor. Esta estructura es particularmente eficiente cuando se desea recuperar todos los libros de un autor específico, ya que una sola consulta a la colección autores devolverá el autor y todos sus libros.

Para encontrar todos los libros de "Gabriel García Márquez", por ejemplo, se haría una consulta como esta:

```
db.autores.find({ "nombre": "Gabriel García Márquez" })
```

Y para encontrar todos los libros en la colección de autores, simplemente se buscaría sin filtros:

```
db.autores.find({})
```

En este caso, el rendimiento de las operaciones de lectura relacionadas con los autores y sus libros puede mejorar, pero al igual que con el modelo anterior, la actualización de cualquier información que se repita en varios documentos (como la nacionalidad del autor si se embebe en cada libro) requeriría actualizaciones múltiples, lo cual es un factor importante a considerar al modelar los datos en MongoDB.

## Modelo de datos de MongoDB, relaciones entre los datos

El esquema de datos en MongoDB es conocido por su flexibilidad, lo que permite a los desarrolladores adaptarlo a las necesidades específicas de su aplicación. Este diseño flexible también se extiende a cómo MongoDB maneja las relaciones entre datos, como se ha puesto de manifiesto en el ejemplo anterior.

Aquí se describen los tipos de relaciones que MongoDB puede modelar y, en cada caso, que alternativas de diseño habría que contemplar

### One-to-One (Uno a Uno)

Para una relación *uno a uno*, donde un documento se relaciona directamente con otro documento, la práctica común es embeber uno dentro del otro. Esto es eficiente cuando los documentos están estrechamente vinculados y se accede a ellos como una unidad única.

Un ejemplo de este caso es el patrón *socio-dirección* donde la dirección de un socio se almacena directamente dentro del documento del socio:

```
{  "_id": ObjectId("507f191e810c19729de860ea"),
   "nombre": "Juan", "apellido1": "Pérez", "apellido2": "Martín",
   "numSocio": 329,
   "direccion": {   "calle": "Avenida de los Conquistadores", "numero": 27,
                    "ciudad": "Salamanca", "CP": "37001"    }
}
```

## One-to-Many (Uno a Muchos Embebido)

Cuando un documento tiene una relación *uno a muchos* con otros documentos, y la cantidad de estos últimos es manejable o no se espera que crezca indefinidamente, se pueden embeber dentro de un array en el documento principal.

Un ejemplo de este caso es el patrón *socio-dirección* donde un socio puede tener varias direcciones, que se almacenan dentro de un array llamado **`direcciones`**:

```
{ "_id": ObjectId("507f191e810c19729de860ea"),
  "nombre": "Juan", "apellido1": "Pérez", "apellido2": "Martín",
  "numSocio": 329,
  "direcciones": [ {    "calle": "Calle Mayor", "numero": 27,
                        "ciudad": "Madrid", "codigoPostal": "28013"     },
                   {    "calle": "Calle del Pez", "numero": 4,
                        "ciudad": "Barcelona", "codigoPostal": "08002"     }
                // ... más direcciones si es necesario
                ]
}
```

## One-to-Many (Uno a Muchos con Referencias)

En algunos casos, es más adecuado mantener los documentos separados y usar referencias al **_id** de los documentos relacionados. Esta es una buena estrategia cuando los documentos relacionados son numerosos o se acceden independientemente con frecuencia.

Para ilustrar una relación *uno a muchos con referencias* en MongoDB, consideremos una editorial y sus libros. Cada libro se relaciona con la editorial que lo publicó a través de una referencia. Aquí hay un ejemplo de cómo se representaría esta relación en documentos de MongoDB.

### Ejemplo de documento para una editorial:

```
{
  "_id": "edSalamandra",
  "ciudad": "Madrid",
  "CIF": "B2832935",
  // Otros campos relevantes para la editorial...
}
```

Este documento representa a la editorial, identificada por un **`_id`** personalizado que es una cadena de texto, en lugar del tradicional ObjectId de MongoDB. Podríamos también usar un ObjectId, pero para fines de demostración se ha optado por una clave más descriptiva.

Ejemplo de documento para un libro:

```
{
  "_id": "HdC1",
  "titulo": "El perfume",
  "ISBN": "348639323598",
  "editorial": "edSalamandra",
  // Otros campos relevantes para el libro...
}
```

En este documento para un libro, hay una clave **`editorial`** que referencia el **`_id`** de la editorial correspondiente. Esto crea una relación *uno a muchos* entre la editorial y sus libros: una editorial puede tener muchos libros, pero cada libro referencia a una sola editorial mediante el campo editorial.

Esta forma de referencia es similar a una clave foránea en una base de datos relacional, pero con la flexibilidad que MongoDB ofrece para el manejo de relaciones. Almacena una referencia a un documento relacionado en lugar de embeber todo el documento. Esto facilita el mantenimiento de los datos, ya que la información de la editorial no necesita repetirse en cada libro, reduciendo así la redundancia de datos y simplificando las actualizaciones. Si la información de la editorial cambia, solo se actualiza el documento de la editorial, no cada documento del libro.

Para obtener información de libros junto con su editorial en MongoDB, se realizarían dos consultas: una para obtener los datos del libro y otra para obtener los detalles de la editorial usando la referencia. En algunas situaciones, especialmente cuando los libros se consultan con frecuencia sin necesidad de información editorial, este enfoque puede ser más eficiente que embeber documentos completos.

## Many-to-Many (Varios a Varios)

Las relaciones *varios a varios* suelen requerir más consideración y hay que valorar la proporción de elementos que hay en cada lado de la relación.

- Pocos-a-Muchos: Si una de las entidades en la relación tiene significativamente menos instancias que la otra, se puede crear un array de referencias en la colección con menos documentos.

  Por ejemplo, a continuación se muestra cómo se verían los documentos para una colección de trabajadores, donde cada trabajador puede estar asignado a varios (pocos) proyectos, y una colección de proyectos, que pueden tener asignados varios (muchos) trabajadores.

  Ejemplo de documento para un trabajador sería:

```
{
  "_id": "T1",
  "nombre": "Juan",
  "apellido": "Pérez",
  "DNI": "12857323F",
  "proyectos": ["P1", "P2"] // Lista de referencias a IDs de proyectos
}
```

Ejemplo de documento para un proyecto sería:

```
{
  "_id": "P1",
  "nombre": "Proyecto 1",
  // Otros campos relevantes para el proyecto
}
```

En este caso, la colección de trabajadores tiene documentos que contienen un campo **`proyectos`** con un array que almacena las referencias a los documentos de la colección de proyectos (identificados por sus **`_id`**).

- Muchos-a-Muchos: Para relaciones equilibradas donde ambas colecciones tienen un número significativo de instancias, podría ser más práctico utilizar una tercera colección intermedia que almacene las relaciones, similar a una tabla de unión en bases de datos relacionales.

  Para ilustrar esta situación se puede considerar el ejemplo anterior pero cuando la relación entre trabajadores y proyectos fuera más equilibrada, optando entonces por crear una tercera colección que actúe como una tabla de unión en bases de datos relacionales. Esta colección tendría documentos que representan cada instancia única de la relación entre un trabajador y un proyecto:

  ```
  {
    "_id": ObjectId("5f50c31e1234567890123456"),
    "trabajador_id": "T1",
    "proyecto_id": "P1"
  }
  ```

  Y en este caso el documento con información del proyecto sería exactamente igual que en el caso anterior y el documento del trabajador ya no tendría el campo proyectos quedando así:

  ```
  {
    "_id": "T1",
    "nombre": "Juan",
    "apellido": "Pérez",
    "DNI": "12857323F"
  }
  ```

  Esta estructura es eficaz cuando hay una cantidad significativa de trabajadores y proyectos, y cada trabajador puede estar asociado con muchos proyectos, y viceversa. Permite un control más granular de las relaciones y puede facilitar operaciones como agregar o eliminar trabajadores de proyectos o generar informes de asignación de proyectos.

## Decisión sobre el modelo de datos a emplear en MongoDB

Cada enfoque tiene sus ventajas y sus desventajas, y **la elección entre ellos debe basarse en factores como la frecuencia de las operaciones de lectura y escritura, el volumen de los datos y la naturaleza de las relaciones entre los datos**. En general, **el modelo de datos en MongoDB debe diseñarse con un enfoque en la forma en que se accederá y se manipulará más frecuentemente la información** para maximizar el rendimiento y la eficiencia.

# Operaciones básicas en MongoDB

Para trabajar con MongoDB y realizar operaciones básicas como la creación, uso y borrado de una base de datos, podemos utilizar la MongoDB Shell, que es una interfaz de línea de comandos interactiva que nos permite realizar operaciones administrativas y de datos.

## Creación de la Base de Datos

Para crear una nueva base de datos, usamos el comando **`use`** seguido del nombre que queramos darle a la base de datos. Si la base de datos no existe, MongoDB la creará tan pronto como se guarde el primer documento en ella.

```
use miLibreria
```

En este caso, **`miLibreria`** es el nombre de la nueva base de datos. No es necesario crear explícitamente la base de datos; simplemente se crea al almacenar el primer documento.

## Utilización de la Base de Datos

Después de seleccionar o crear la base de datos con el comando **`use`**, puedes comenzar a trabajar con ella a través de la variable **`db.`** Esta variable es una referencia a la base de datos actualmente seleccionada.

Por ejemplo, para insertar un nuevo documento en una colección llamada **`libros`**, usarías el siguiente comando:

```
db.libros.insert({ titulo: "Cien años de soledad",
autor: "Gabriel García Márquez" })
```

Cada comando que quieras ejecutar sobre la base de datos se iniciará con **`db`**, seguido del nombre de la colección y el comando correspondiente.

## Borrado de la Base de Datos

Si necesitas borrar la base de datos que has seleccionado, puedes hacerlo con el comando **`db.dropDatabase().`** Este comando eliminará la base de datos actualmente seleccionada y todos los datos contenidos en ella.

```
use miLibreria
db.dropDatabase()
```

Este comando es irreversible y debe usarse con precaución, ya que eliminará todos los datos almacenados en la base de datos **`miLibreria`**.

## Creación de Colecciones

En MongoDB, las colecciones son similares a las tablas en las bases de datos relacionales y sirven para organizar los documentos. A continuación, se explica cómo se pueden crear colecciones dentro de una base de datos de MongoDB.

### Creación Implícita

MongoDB no requiere que crees explícitamente una colección antes de usarla. Una colección se crea automáticamente cuando insertas el primer documento. Por ejemplo,

si queremos crear una colección llamada usuarios e insertar un documento en ella, podríamos hacer lo siguiente:

```
db.usuarios.insert({
nombre: "Juan",
apellido: "Pérez",
edad: 30
})
```

Al ejecutar este comando, si la colección **`usuarios`** no existe, MongoDB la crea y añade el documento proporcionado.

### Creación Explícita

En ocasiones, puedes querer crear una colección con parámetros específicos, como el tamaño máximo o el tiempo de expiración de los documentos. Para estos casos, se utiliza el método **`db.createCollection`**. Por ejemplo:

```
db.createCollection("mensajeslog", {
  capped: true,
  size: 100000,
  max: 5000
 })
```

Este comando crea una colección **`mensajeslog`** que es de tipo "**`capped`**", lo que significa que tiene un tamaño máximo y un número máximo de documentos. Una vez que la colección alcanza el tamaño máximo, MongoDB comienza a sobrescribir los documentos más antiguos. En concreto, el significado de cada uno de los parámetros anteriores es:

- La colección es de tamaño fijo (**`capped: true`**).
- Tiene un tamaño máximo de 100,000 bytes (**`size: 100000`**).
- Solo puede contener un máximo de 5,000 documentos (**`max: 5000`**).

Es importante tener en cuenta que las colecciones "*capped*" son útiles para casos de uso como el almacenamiento de logs, donde sólo te interesa mantener una cantidad fija de los datos más recientes.

Con estos métodos, MongoDB proporciona una manera flexible y eficiente de trabajar con colecciones, permitiendo a los desarrolladores adaptar la base de datos a las necesidades específicas de su aplicación.

Para más información sobre las opciones disponibles al crear colecciones, puedes consultar la documentación oficial de MongoDB: db.createCollection documentation.

# Operaciones CRUD en MongoDB

Las operaciones CRUD son las operaciones básicas de datos realizadas en las bases de datos. CRUD es un acrónimo de "*Crear, Leer, Actualizar, Eliminar*" (*Create, Read, Update, Delete*). En MongoDB, estas operaciones se realizan a través de la MongoDB Shell o de un driver en el lenguaje de programación de tu elección.

## Crear datos - *Insert*

En MongoDB, la operación de inserción de documentos en una colección se puede realizar con los métodos **`insertOne()`** para un solo documento o **`insertMany()`** para múltiples documentos. Si no se proporciona un valor para el campo **`_id`** al insertar un documento, MongoDB generará automáticamente un valor único de tipo **`ObjectId`**.

Aquí hay un ejemplo de cómo insertar documentos en una colección utilizando MongoDB:

### Insertar un solo documento - `insertOne`

```
db.libros.insertOne({
  titulo: "La sombra del viento",
  autor: "Carlos Ruiz Zafón",
  año: 2001
})
```

Si no especificas un **`_id`**, MongoDB asignará uno automáticamente y, por tanto, el documento que realmente se almacena en la colección sería:

```
{
  "_id": ObjectId("5f50c31e1234567890123456"),
  titulo: "La sombra del viento",
  autor: "Carlos Ruiz Zafón",
  año: 2001
}
```

### Insertar múltiples documentos - `insertMany`

```
db.libros.insertMany([
  {
    titulo: "El juego del ángel",
    autor: "Carlos Ruiz Zafón",
    año: 2008
  },
  {
    titulo: "El prisionero del cielo",
    autor: "Carlos Ruiz Zafón",
    año: 2011
  }
])
```

En este caso, cada documento insertado que no tenga un campo **`_id`** proporcionado recibirá un **`ObjectId`** generado automáticamente por MongoDB y, por tanto, los documentos que realmente se almacenan en la colección serían:

```
[
 {
  "_id": ObjectId("5f50c32012345678901234567"),
  titulo: "El juego del ángel",
  autor: "Carlos Ruiz Zafón",
  año: 2008
 },
 {
  "_id": ObjectId("5f50c32112345678901234568"),
  titulo: "El prisionero del cielo",
  autor: "Carlos Ruiz Zafón",
  año: 2011
 }
]
```

### Carga masiva de datos - `mongoimport`

MongoDB ofrece varias opciones para la carga masiva de datos, lo que es particularmente útil cuando se necesita importar grandes volúmenes de información de manera eficiente. Para estos casos, MongoDB proporciona herramientas específicas como **`mongoimport`** y métodos como **`insertMany()`**, visto antes.

La herramienta **`mongoimport`** está diseñada para la importación de datos en formato JSON, CSV o TSV directamente desde la línea de comandos a una colección en una base de datos de MongoDB. Es ideal para la carga inicial de datos o para la transferencia de datos entre sistemas.

Un ejemplo de uso de **`mongoimport`** sería con el siguiente comando en la *shell* que importará datos desde el archivo **`libros.json`** a la colección **`libros`** en la base de datos **`miLibreria`**.

```
mongoimport --db miLibreria --collection libros --file /path/to/libros.json
```

### Carga masiva de datos desde MongoDB Atlas

MongoDB Atlas (https://www.mongodb.com/atlas) es un servicio de base de datos en la nube completamente administrado por MongoDB, Inc. Ofrece a los usuarios la capacidad de alojar sus bases de datos en una plataforma automatizada, escalable y segura en la nube, sin la necesidad de manejar la infraestructura subyacente. Atlas facilita la implementación, operación y escalabilidad de las bases de datos de MongoDB en los principales proveedores de la nube, como AWS, Azure y Google Cloud Platform.

Con MongoDB Atlas, puedes utilizar la funcionalidad de Data Import para cargar datos directamente desde la consola de Atlas. Esta característica admite la importación de archivos JSON y CSV a tus clústeres de Atlas. Para realizar una carga masiva de datos en Atlas, sigues estos pasos generales:

- Preparación de datos: Prepara tus datos en un archivo JSON o CSV. Asegúrate de que estén formateados correctamente según las especificaciones de MongoDB.
- Consola de Atlas: Inicia sesión en tu cuenta de MongoDB Atlas y navega a la interfaz de tu clúster.
- Importación de Datos: Utiliza las opciones de importación de datos para especificar la ubicación de tus archivos de datos, nombre de la colección, etc. y

comienza el proceso de importación. Atlas proporcionará una interfaz de usuario para configurar y ejecutar la tarea de importación.

- Monitoreo: Puedes monitorear el progreso de la importación en tiempo real desde la consola de Atlas.

## Actualización de datos - `update`

Las operaciones de actualización son fundamentales para mantener los datos actualizados y relevantes. En MongoDB, la operación de actualización se puede realizar mediante diversos métodos, como **`updateOne`**, **`updateMany`**, y **`findOneAndUpdate`**. Estos métodos permiten modificar los documentos existentes en una colección. El operador **`$set`** es comúnmente usado en estas operaciones para especificar los nuevos valores de los campos que se quieren actualizar.

### Actualización con $set

El operador **`$set`** es usado para especificar los campos que se deben actualizar con nuevos valores. Si el campo no existe, **`$set`** lo añadirá con el valor especificado.

Ejemplo de actualización de un solo documento con **`updateOne`**:

```
db.libros.updateOne(
  { titulo: "1984" },  // Condición del documento a actualizar
  { $set: { autor: "George Orwell", año: 1949 } }  // Valores nuevos
)
```

Este comando actualizará el primer documento en la colección libros que tenga el **`titulo "1984`**" y establecerá o actualizará el **`autor`** y el **`año`** de publicación.

Ejemplo de actualización de varios documentos con **`updateMany`**:

```
db.libros.updateMany(
  { autor: "Gabriel García Márquez" }, // Filtro documentos actualizar
  { $set: { genero: "Realismo mágico" } } // Valor nuevo
)
```

Al utilizar updateMany, MongoDB automáticamente aplica la operación de actualización a todos los documentos que coinciden con el filtro/condición proporcionada. Este comando actualiza todos los documentos en la colección **`libros`** donde el **`autor`** es "**`Gabriel García Márquez`**", añadiendo o actualizando el campo **`genero`**.

### Creación de Documentos si no Existen (Upsert)

La opción **`upsert`** se utiliza para crear un nuevo documento si no hay ninguno que coincida con el criterio de búsqueda.

```
db.libros.updateOne(
  { titulo: "Cien años de soledad" },
  { $set: { autor: "Gabriel García Márquez", año: 1967 } },
  { upsert: true }
)
```

Si no existe un documento con el título "**`Cien años de soledad`**", este comando creará uno nuevo con los campos **`autor`** y **`año`**.

## Borrado de datos – **`delete`** y **`drop`**

En MongoDB, para borrar documentos de una colección, se pueden utilizar los métodos **`deleteOne`**, **`deleteMany`**. Mientras que **`drop`** se usa para eliminar una colección completa.

- **`deleteOne`**: Elimina el primer documento que coincide con los criterios de búsqueda en la colección.
- **`deleteMany`**: Elimina todos los documentos que coinciden con los criterios de búsqueda en la colección.

Ejemplo de borrado de un documento con **`deleteOne`**:

Si tienes una colección libros y quieres eliminar un único libro por su título, puedes usar el método **`deleteOne`**. Este comando eliminará el primer libro que encuentre con el título especificado

```
db.libros.deleteOne({ titulo: "La sombra del viento" })
```

Este comando buscará en la colección **`libros`** el primer documento con el título "**`La sombra del viento`**" y lo eliminará. Si hubiera más de un libro con ese título, solo se eliminará uno, el primero en la colección.

Ejemplo de borrado de varios documentos con **`deleteMany`**:

**`deleteMany`** se utiliza para eliminar todos los documentos que coinciden con un criterio específico. Si quieres eliminar todos los libros de un autor en particular, este es el método que usarías.

```
db.libros.deleteMany({ autor: "Carlos Ruiz Zafón" })
```

Este comando eliminará todos los documentos en la colección **`libros`** que tengan como autor a "`Carlos Ruiz Zafón`".

Ejemplo de borrado de una colección con **`drop`**:

El método **`drop`** se usa para eliminar una colección completa, incluyendo todos sus documentos e índices. Si decides que necesitas eliminar toda la colección **`libros`**, por ejemplo, durante una reestructuración de tu base de datos o si la colección ya no es necesaria, puedes usar **`drop`**.

```
db.libros.drop()
```

Al ejecutar este comando, la colección entera **`libros`** se eliminará de la base de datos. Esto es una acción inmediata y no se pueden recuperar los datos una vez que se ha ejecutado el método **`drop`**, a menos que tengas un respaldo de la base de datos.

# Consultas en MongoDB

## Introducción

En cualquier sistema de gestión de bases de datos, las consultas son esenciales porque son el medio por el cual se recupera la información. Esta capacidad de recuperación es crucial para el análisis de datos, la toma de decisiones empresariales, la generación de informes y, en última instancia, cualquier operación que requiera información precisa y oportuna de la base de datos.

Un aspecto distintivo de las consultas en MongoDB es que, dada su naturaleza NoSQL y orientada a documentos, no requieren un esquema fijo. Esto significa que las consultas pueden adaptarse dinámicamente a los datos disponibles, lo que es ideal para manejar datos heterogéneos y para aplicaciones que evolucionan rápidamente en términos de estructura de datos.

MongoDB se distingue en este ámbito con su método **`find`**(), que es la piedra angular de las operaciones de consulta. Este método ofrece una forma flexible y poderosa de buscar y recuperar documentos dentro de una colección. Utilizando **`find`**(), puedes especificar criterios de búsqueda detallados y estructurados, incluyendo operadores de comparación, lógicos, y de elementos de array, para afinar la selección de documentos.

Además, MongoDB introduce un concepto poderoso conocido como la "Aggregation Pipeline" que se verá en la siguiente sección. Esta es una estructura de procesamiento de datos que permite pasar documentos a través de una secuencia de operaciones, conocidas como "etapas", donde el resultado de una etapa es introducido en la siguiente. Cada etapa transforma los documentos de alguna manera: filtrando, agregando, proyectando, ordenando, entre otras operaciones. Este concepto de "pipe" permite la construcción de consultas y transformaciones de datos muy sofisticadas que serían difíciles o imposibles de realizar con una única consulta estándar.

En resumen, las consultas son fundamentales en la administración de bases de datos, y MongoDB ofrece un enfoque único que aprovecha la flexibilidad de su modelo de datos con una suite de herramientas de consulta altamente potentes y flexibles, incluyendo la Aggregation Pipeline, lo que hace que sea una solución robusta y deseable en el paisaje de la gestión de datos moderna.

## Método `find()`

### Introducción

El comando **`find()`** en MongoDB es uno de los aspectos más importantes y potentes del trabajo con esta base de datos NoSQL. A través de **`find()`**, los usuarios pueden ejecutar consultas para recuperar documentos específicos de sus colecciones, ofreciendo una flexibilidad y precisión extraordinarias para el manejo de datos. Este comando se asemeja al **`SELECT`** de SQL en bases de datos relacionales, pero va más allá en cuanto a su capacidad para manejar consultas en datos estructurados y semiestructurados.

Cuando se invoca **`find()`**, se pueden especificar condiciones de búsqueda que filtran los documentos según los campos y los valores que contienen. Puede ser tan simple como buscar documentos que tengan un campo con un valor específico o tan complejo como

realizar búsquedas que incluyan varios campos con múltiples criterios, aprovechando los operadores de comparación y lógicos de MongoDB. Además, **`find()`** puede ser refinado con proyecciones que limitan los campos devueltos, lo que resulta en una operación más eficiente especialmente cuando se trabaja con documentos grandes o se requiere un alto rendimiento de la red.

Para modificar aún más el resultado, los modificadores como **`sort()`**, **`limit()`** y **`skip()`** se pueden encadenar al comando **`find()`**. Estos modificadores ajustan el conjunto de resultados finales, ordenándolos, limitando la cantidad de documentos devueltos o saltando un número determinado de ellos, lo que permite a los usuarios recuperar exactamente lo que necesitan.

**`find()`** también es notablemente poderoso cuando se trata de trabajar con arrays y subdocumentos. Operadores como **`$elemMatch`** y **`$size`** brindan control sobre cómo se pueden buscar y filtrar los datos anidados.

En resumen, **`find()`** es una herramienta esencial para cualquier persona que trabaje con MongoDB, proporcionando la base sobre la cual se pueden construir consultas para navegar y extraer datos de una de las bases de datos MongoDB

## Primeros pasos

Comenzando con lo más básico, el comando **`find()`** en MongoDB puede utilizarse sin ninguna condición para recuperar todos los documentos dentro de una colección. Esto es útil cuando se quiere obtener una visión general de los datos o cuando se necesita procesar todos los documentos de una colección.

La sintaxis del comando **`find()`** es:

```
db.coleccion.find({filtro_documentos})
```

### Ejemplo Básico: Sin Condiciones

Si tienes una colección llamada **`libros`** y deseas recuperar todos sus documentos, la consulta sería:

```
db.libros.find()
```

o también:

```
db.libros.find({})
```

Este comando le dice a MongoDB que busque en la colección libros y recupere todos los documentos. El **`{}`** es un filtro de búsqueda vacío que no aplica ninguna condición.

### Ejemplo con Condiciones: Búsqueda por Autor

Si quieres encontrar libros de un autor específico, puedes añadir una condición al filtro de búsqueda:

```
db.libros.find({autor: "Gabriel García Márquez"})
```

Aquí, **`{autor: "Gabriel García Márquez"}`** es el filtro que especifica la condición para la búsqueda: queremos solo los documentos donde el campo **`autor`** es igual a "**`Gabriel García Márquez`**".

### Proyecciones

Las proyecciones permiten especificar qué campos quieres que se incluyan o excluyan en los documentos que devuelve la consulta. Para emplear proyecciones con el comando find() se emplea la siguiente sintaxis:

```
db.coleccion.find({filtro_documentos},{proyecciones})
```

Por ejemplo, si solo estás interesado en los títulos de los libros y los años de publicación, podrías escribir:

```
db.libros.find({autor:"Gabriel García Márquez"}, {titulo: 1, año: 1, _id: 0 })
```

En esta consulta, **`{ titulo: 1, año: 1, _id: 0 }`** es la proyección. El **`1`** indica que queremos incluir esos campos en los resultados, y el **`_id: 0`** indica que no queremos que el campo **`_id`** se incluya en los resultados.

Esta secuencia de ejemplos muestra cómo **`find()`** puede ser tan simple o tan complejo como lo requieran tus necesidades de consulta. Comenzando sin condiciones te da todo, añadir una condición filtra por esa condición, y las proyecciones te permiten centrarte en la información relevante, optimizando así la transferencia de datos y la claridad de los resultados.

### Modificadores

Los modificadores se usan para cambiar el resultado de la búsqueda. Por ejemplo, puedes ordenar los resultados, limitar la cantidad de documentos devueltos, o saltar un número específico de documentos.

Por ejemplo, para obtener los libros de un autor y ordenarlos por año de publicación, podrías usar:

```
db.libros.find({ autor: "Gabriel García Márquez" }).sort({ año: 1 })
```

En este ejemplo, **`.sort({año: 1})`** ordena los documentos en orden ascendente según el campo **`año`**.

### Ejemplo Completo

```
db.libros.find(
  { autor: "Isabel Allende" },   // Condiciones
  { titulo: 1, año: 1, _id: 0 }  // Proyecciones
).limit(5).sort({ año: -1 })     // Modificadores: Limita a 5 documentos
                                 // y los ordena por año descendente
```

Este comando buscará los libros de "**`Isabel Allende`**", pero sólo devolverá los campos **`titulo`** y **`año`** de los primeros 5 libros, ordenados de más reciente a más antiguo.

La documentación oficial de MongoDB proporciona una guía completa y ejemplos adicionales de cómo utilizar **`find()`** y sus diversas opciones para realizar consultas: *Query Documents in MongoDB* (https://docs.mongodb.com/manual/tutorial/query-documents/) .

Estas son las bases de cómo realizar consultas de lectura en MongoDB, lo que permite una gran flexibilidad y potencia al recuperar datos de tu base de datos.

## Sintaxis general del comando `find()`

La sintaxis general del comando **`find`**() en MongoDB se compone de dos partes principales: el filtro de consulta y la proyección de campos. Aquí está la estructura básica:

```
db.coleccion.find(filtro, proyeccion)
```

Donde:

- **`db`**: Es el prefijo que representa la base de datos actualmente seleccionada.
- **`coleccion`**: Es el nombre de la colección dentro de la base de datos donde se realizará la búsqueda.
- **`find`**(): Es el método que ejecuta la operación de búsqueda.
- **`filtro`** (opcional): Es un documento que especifica los filtros o condiciones de la búsqueda. Si se omite o se pasa un documento vacío **`{}`**, **`find`**() devolverá todos los documentos de la colección.
- **`proyeccion`** (opcional): Es un documento que especifica los campos que se deben incluir o excluir en los documentos que devuelve la consulta.

Además, **`find`**() devuelve un cursor a los documentos resultantes, y puedes encadenar otros métodos, como **`sort`**(), **`limit`**(), y **`skip`**(), al cursor para modificar aún más el resultado:

```
db.collection.find({ campo: valor }).sort({ campo: 1 }).limit(n).skip(m)
```

Esta es la base para trabajar con el comando **`find`**() en MongoDB, lo cual es fundamental para cualquier operación de recuperación de datos en esta base de datos.

## Condiciones - operadores de comparación

En MongoDB, las consultas pueden incluir una variedad de condiciones que te permiten filtrar documentos de forma precisa. Usando operadores especiales dentro del método **`find()`**, se puede construir un filtro para especificar qué datos se están buscando en la colección de la base de datos.

Al construir las condiciones para una consulta utilizando el comando **`find()`** en MongoDB, es fundamental comenzar con los operadores de comparación. Estos operadores son las herramientas esenciales que permiten filtrar y recuperar datos con precisión. Permiten comparar el valor de un campo dentro de tus documentos con un valor de consulta especificado, para que puedas encontrar exactamente lo que estás buscando.

La sintaxis para usar estos operadores de comparación es:

```
{ campo: { operador: valor } }
```

Donde:

- campo es el nombre del campo en los documentos contra el cual se quiere realizar la comparación.
- operador es uno de los operadores de comparación de MongoDB, precedido por un signo de dólar (**`$`**). Algunos de los operadores de comparación más comunes

incluyen **`$eq`** (igual), **`$gt`** (mayor que), $**`gte`** (mayor o igual que), **`$lt`** (menor que), **`$lte`** (menor o igual que), y **`$ne`** (no igual).

- valor es el valor contra el cual se está comparando el contenido del campo.

La siguiente tabla muestra la lista de operadores de comparación:

| Operador | Significado en Inglés | Descripción |
|---|---|---|
| `$eq` | Equals | Igual a |
| `$gt` | Greater Than | Mayor que |
| `$gte` | Greater Than or Equal | Mayor o igual que |
| `$lt` | Less Than | Menor que |
| `$lte` | Less Than or Equal | Menor o igual que |
| `$ne` | Not Equal | Diferente de |
| `$in` | In | Dentro de un conjunto de valores |
| `$nin` | Not In | No dentro de un conjunto de valores |

Vamos a ver algunos ejemplos con la colección libros para entender cómo funcionan los operadores de comparación:

### Igualdad: $eq

El operador **`$eq`** (*Equals - Igual a*) se utiliza para buscar documentos donde el valor de un campo es igual al valor especificado. Este operador es implícito dado que si solo se proporciona el valor, no es necesario especificar **`$eq`**.

Por ejemplo, los dos siguientes comandos son equivalente y buscan libros cuyo autor sea exactamente "Gabriel García Márquez":

```
db.libros.find({ autor: "Gabriel García Márquez" }) // sin $eq

db.libros.find({ autor: { $eq: "Gabriel García Márquez" } }) // con $eq
```

### Mayor que $gt y Mayor o Igual que $gte

El operador **`$gt`** (*Greater Than - Mayor que*) y el operador **`$gte`** (Greater Than or Equal - Mayor o igual que) se utilizan para buscar documentos donde el valor de un campo es *mayor que*, o *mayor o igual que*, el valor especificado.

Por ejemplo, el comando a utilizar si deseamos recuperar los libros publicados después de 1950:

```
db.libros.find({ año: { $gt: 1950 } })
```

o si deseamos encontrar los libros publicados en 1950, o después de 1950:

```
db.libros.find({ año: { $gte: 1950 } })
```

### Menor que $lt y menor o igual que $lte

Los operadores **`$lt`** (*Less Than - Menor que*) y **`$lte`** (Less Than or Equal - Menor o igual que) se usan para condiciones donde el valor de un campo *es menor que,* o *menor o igual que,* el valor especificado.

Por ejemplo, el comando a utilizar si deseamos recuperar los libros publicados antes de 2000:

```
db.libros.find({ año: { $lt: 2000 } })
```

o si deseamos encontrar los libros publicados en 2000, o antes de 2000:

```
db.libros.find({ año: { $lte: 2000 } })
```

### Diferente a $ne

El operador **`$ne`** (*Not Equal - Diferente de*) se utiliza para encontrar documentos donde el valor de un campo es diferente al valor especificado.

Por ejemplo, el comando a utilizar si deseamos recuperar los libros que no son de Gabriel García Márquez:

```
db.libros.find({ autor: { $ne: "Gabriel García Márquez" } })
```

### Pertenece a $in y No Pertenece a $nin

Con el operador **`$in`** (*In - Dentro de, pertenece a*), puedes buscar documentos donde el valor de un campo es igual a cualquiera de los valores listados en un array. Por otro lado, `$nin` (*Not In - No está dentro de, no pertenece a*) encuentra los documentos donde el valor del campo no coincide con ningún valor del array especificado.

Por ejemplo, el comando a utilizar si deseamos recuperar los libros de García Márquez o Isabel Allende:

```
db.libros.find({autor:{ $in: ["Gabriel García Márquez", "Isabel Allende"] } })
```

o si deseamos encontrar los libros que no son ni de García Márquez ni de Isabel Allende:

```
db.libros.find({ autor: { $nin: ["Gabriel García Márquez", "Isabel Allende"] } })
```

## Condiciones - operadores lógicos

Los operadores lógicos en MongoDB te permiten combinar múltiples condiciones de consulta, lo que te da un gran control sobre cómo deseas que los documentos coincidan con tu consulta. A continuación se muestra una tabla con los operadores lógicos principales y sus descripciones:

| Operador | Descripción |
|---|---|
| `$and` | Permite combinar múltiples condiciones que deben ser todas ciertas para que un documento se incluya en el resultado. |
| `$or` | Permite que un documento cumpla al menos una de las condiciones especificadas. |
| `$not` | Invierte el efecto de una condición de consulta y devuelve los documentos que no coinciden con la condición. |
| `$nor` | Combina múltiples condiciones y devuelve los documentos que no coinciden con ninguna de ellas. |

Los operadores lógicos sobre dos o más condiciones, se aplican a un array cuyos elementos son las condiciones. Por tanto, la sintaxis sería:

```
{ $and: [ { condicion1 }, { condicion2 }, ... ] }
{ $or: [ { condicion1 }, { condicion2 }, ... ] }
{ $nor: [ { condicion1 }, { condicion2 }, ... ] }
```

El operador $not que se emplea para negar una condición especificada, su sintaxis si se aplica con otro operador sería:

```
campo: {$not: {operador: valor}}
```

Veamos algunos ejemplos con la colección libros para entender cómo funcionan los operadores lógicos.

Por ejemplo, si quiero buscar libros de “Gabriel García Márquez” escritos después de 1967, utilizaría el comando:

```
db.libros.find({
  $and: [
   { autor: "Gabriel García Márquez" },
   { año: { $gt: 1967 } }
  ]
 })
```

Si ahora quiero buscar libros de “Gabriel García Márquez” o "Isabel Allende", utilizaría el comando:

```
db.libros.find({
  $or: [
   { autor: "Gabriel García Márquez" },
   { autor: "Isabel Allende" }
  ]
 })
```

Si ahora quiero buscar libros cuyo precio no sea mayor de 20:

```
db.libros.find({
  precio: { $not: { $gt: 20 } }
 })
```

Ahora una búsqueda de documentos donde el autor no es "Gabriel García Márquez" o que el año no es menor que 1900, es decir, documentos que cumplan una de las dos condiciones, pero únicamente una:

```
db.libros.find({
  $nor: [
    { autor: "Gabriel García Márquez" },
    { año: { $lt: 1900 } }
  ]
})
```

Estos operadores son poderosos porque permiten construir expresiones lógicas complejas, proporcionando la capacidad de realizar consultas que se ajusten a criterios específicos y variados. Con el uso de operadores lógicos, las consultas pueden ser tan simples o tan complejas como sea necesario.

## Condiciones – Anidamiento

El anidamiento en las consultas de MongoDB es una técnica poderosa que permite evaluar múltiples condiciones en una única operación **`find`**(). Este enfoque es similar a cómo podríamos usar paréntesis en una expresión matemática para establecer el orden y la precedencia de las operaciones; en MongoDB, usamos la estructura de documentos y arrays para obtener el mismo efecto.

### Anidamiento Básico

El anidamiento básico se realiza utilizando subdocumentos en los criterios de búsqueda. Por ejemplo, si tienes una colección de personas y quieres encontrar a las personas que viven en "Nueva York" y tienen más de 30 años, podrías escribir algo como:

```
db.personas.find({
  "direccion.ciudad": "Nueva York",
  edad: { $gt: 30 }
})
```

### Anidamiento con Operadores Lógicos

El anidamiento se vuelve más complejo y potente cuando utilizas operadores lógicos como $**`and`**, $**`or`**, $**`not`** y $**`nor`**. Estos operadores permiten combinar varias condiciones.

Por ejemplo, si buscas documentos que deben cumplir dos o más condiciones simultáneamente, puedes anidar estas condiciones dentro de un operador $and:

```
db.personas.find({
  $and: [
    { "direccion.ciudad": "Nueva York" },
    { edad: { $gt: 30 } },
    { "profesion": "Ingeniero" }
  ]
})
```

Este comando devolverá documentos de la colección personas donde la ciudad es "Nueva York", la edad es mayor que 30 y la profesión es "Ingeniero".

### Anidamiento Complejo

Puedes llevar el anidamiento aún más lejos combinando múltiples operadores lógicos. Esto permite realizar consultas que incluyen una serie de condiciones "y/o" más complejas.

```
db.personas.find({
  $or: [
    {
      $and: [
        { "direccion.ciudad": "Nueva York" },
        { edad: { $gt: 30 } }
      ]
    },
    {
      $and: [
        { "direccion.ciudad": "Los Ángeles" },
        { edad: { $lte: 25 } }
      ]
    }
  ]
})
```

Aquí, la consulta devolverá personas que (viven en "Nueva York" Y tienen más de 30 años) O (viven en "Los Ángeles" Y tienen 25 años o menos).

### Anidamiento con Arrays y $elemMatch

Para colecciones que contienen arrays, el operador $**elemMatch** permite especificar múltiples criterios en los elementos de un array, lo que agrega otro nivel de anidamiento. Este operador se ve más adelante en este documento.

Como ejemplo de este operador, supongamos que la colección **personas** contiene documentos con un array de nombre **aficiones** con los campos **tipo** y **nivel**. Si se desea buscar personas cuyas aficiones incluyan algún deporte con un nivel de habilidad de 5 o superior.

```
db.personas.find({
  "aficiones": {
    $elemMatch: {
      tipo: "deporte",
      nivel: { $gte: 5 }
    }
  }
})
```

### Anidamiento en la práctica

El anidamiento de condiciones es especialmente útil en situaciones de la vida real donde los datos son complejos y las preguntas que necesitas responder son detalladas. Por ejemplo, si necesitas encontrar todos los libros en una librería que fueron publicados después de 1990 pero que no son de los géneros "Ciencia Ficción" o "Fantasía", o todos los productos en un inventario que están en cierto rango de precios y tienen una

calificación de reseñas promedio alta pero no están descontinuados, el anidamiento es la herramienta que te permite construir esas consultas.

## Condiciones – Existencia de campo ($exist)

El operador $**exists** en MongoDB es especialmente útil en bases de datos "*schemaless*" (sin esquema fijo), como es el caso de MongoDB, donde diferentes documentos dentro de la misma colección pueden tener estructuras diferentes. Esto significa que ciertos campos pueden estar presentes en algunos documentos, pero no en otros. El operador $**exists** permite realizar consultas basadas en la existencia (o ausencia) de estos campos.

### Uso de $exists

El operador $**exists** acepta un valor booleano (**true** o **false**) que indica si MongoDB debe buscar documentos donde el campo especificado exista o no.

Por ejemplo, si se desea encontrar documentos en la colección **libros** que SÍ tengan el campo **resumen**, se emplearía el comando:

```
db.libros.find({ "resumen": { $exists: true } })
```

Sin embargo, si se desea encontrar documentos en la colección **libros** que NO tengan el campo **resumen**, se emplearía el comando:

```
db.libros.find({ "resumen": { $exists: false } })
```

### Importancia de $exists en bases de datos *Schemaless*

La flexibilidad de MongoDB para permitir diferentes estructuras de documentos en la misma colección es una de sus características más potentes, facilitando el modelado de datos complejos y variados. Sin embargo, esta flexibilidad también introduce la necesidad de poder consultar documentos basándose en la presencia o ausencia de ciertos campos.

El operador $**exists** es crucial para manejar estos escenarios, permitiendo a los desarrolladores:

- Realizar limpiezas de datos o migraciones donde necesitas identificar documentos que carecen de ciertos campos.
- Implementar lógicas condicionales en las consultas, donde el procesamiento o las acciones posteriores dependen de la presencia de ciertos datos.
- Analizar la estructura de los datos existentes, útil tanto para la depuración como para la comprensión del esquema de datos emergente.

Aunque $**exists** es poderoso, su uso debe ser considerado cuidadosamente, especialmente en colecciones grandes, ya que la búsqueda de campos no indexados puede ser costosa en términos de rendimiento. Cuando sea posible, estructurar los documentos de manera que se pueda evitar el uso excesivo de $**exists**, o considerar el uso de índices en campos que son comúnmente consultados para existencia, puede ayudar a mantener un buen rendimiento de la base de datos.

## Condiciones – Tipo de un campo ($type)

El operador **`$type`** en MongoDB se utiliza para buscar documentos en los que el valor de un campo es de un tipo de dato específico. Este operador es particularmente útil en una base de datos *Schemaless* como MongoDB, donde los campos pueden tener diferentes tipos de datos en diferentes documentos, o donde el tipo de dato de un campo puede cambiar con el tiempo.

El operador **`$type`** permite especificar un tipo de dato BSON en la consulta, y MongoDB devolverá solo aquellos documentos en los que el campo especificado es del tipo deseado.

Por ejemplo, si se desea encontrar documentos en la colección **`libros`** en los que el campo **`'precio'`** es de tipo **`'double'`**, se utilizaría el comando:

```
db.libros.find({ precio: { $type: "double" } })
```

O si lo que se desea es encontrar los documentos donde el campo **`'fecha_publicacion'`** es de tipo **`'date'`**, se utilizaría el comando:

```
db.libros.find({ fecha_publicacion: { $type: "date" } })
```

Los tipos de datos BSON incluyen, entre otros, **`string`**, **`double`**, **`date`**, **`array`**, **`objectId`**, etc. Para $**`type`**, puedes especificar el tipo de dato tanto por nombre como por número asociado al tipo en BSON.

### Utilidad de $type

El operador $**`type`** es especialmente útil en los siguientes casos:

- Validación de datos: Asegurarse de que los datos tienen el formato correcto antes de realizar operaciones como actualizaciones o cálculos.
- Limpieza y migración: Identificar documentos que necesitan corrección o conversión de tipos de datos durante las tareas de mantenimiento de la base de datos.
- Análisis de esquema: Comprender cómo los datos están estructurados en la base de datos, lo cual es útil cuando se hereda una base de datos o se trabaja con datos generados por usuarios o procesos sobre los que no se tiene control completo.
- Optimización de índices: Crear índices más efectivos al comprender los tipos de datos de los campos sobre los que se indexa, lo cual puede tener un impacto significativo en el rendimiento.

Usar $**`type`** puede ayudar a garantizar que las operaciones que dependen de un determinado formato de dato se realicen correctamente, evitando errores en la aplicación y problemas de lógica. Además, el conocimiento del tipo de dato es crucial para las optimizaciones de rendimiento y las estrategias de indexación en MongoDB.

## Condiciones – Sobre arrays ($all, $elemMatch, $size)

Trabajar con arrays en MongoDB implica una serie de operadores que permiten realizar búsquedas y comparaciones dentro de estos. Los operadores $**`all`**, $**`elemMatch`** y $**`size`** son específicamente útiles cuando se quieren realizar operaciones de consulta en campos que son arrays.

### $all

El operador $**all** se utiliza cuando necesitas encontrar documentos donde un campo de tipo array contiene todos los elementos especificados, independientemente del orden en que aparezcan.

Por ejemplo, si se desea encontrar todos los documentos en la colección **libros** donde el campo **generos** incluye tanto "**ficción**" como "**misterio**", se emplearía el comando:

```
db.libros.find({ generos: { $all: ["ficción", "misterio"] } })
```

### $elemMatch

El operador $**elemMatch** se utiliza para buscar documentos donde al menos un elemento del array cumple con múltiples criterios de consulta especificados. Es especialmente útil cuando los arrays contienen subdocumentos y deseas que se cumpla más de una condición en el mismo elemento.

Por ejemplo, se desea encontrar los documentos de la colección **libros** donde haya al menos una **reseña** que tenga una **calificacion** mayor o igual a **5** y que además haya sido escrita por el **lector** identificado como "**John Doe**".

```
db.libros.find({
  reseñas: {
    $elemMatch: {
      calificacion: { $gte: 5 },
      lector: "John Doe"
    }
  }
})
```

### $size

El operador $**size** te permite buscar documentos donde el campo array tiene un número específico de elementos.

Por ejemplo, se desea encontrar los documentos de la colección **libros** donde en el array géneros tenga definidos tres elementos, se utilizaría el comando:

```
db.libros.find({ generos: { $size: 3 } })
```

Estos operadores son de gran utilidad para interactuar con arrays dentro de tus documentos MongoDB:

### Resumen:

- **$all** es útil cuando se busca documentos con un array que contenga un conjunto de elementos, sin importar el orden.
- **$elemMatch** se utiliza en búsquedas de documentos con arrays con una coincidencia basada en más de una condición en un elemento individual.
- **$size** cuando interesa consultar la cantidad de elementos en el array, como filtrar colecciones basadas en el número de atributos o características.

Utilizando estos operadores, se pueden construir consultas que permitan aprovechar la estructura de documentos flexibles y dinámicos que MongoDB maneja.

## Modificadores

Los modificadores en MongoDB son opciones adicionales que puedes aplicar después de una consulta **`find`**() para alterar el conjunto de resultados o la forma en que se procesa la consulta. Afectan directamente a la salida de la consulta, proporcionando control sobre el ordenamiento, la cantidad de documentos devueltos, el número de documentos a omitir y otras funcionalidades avanzadas.

Los modificadores se aplican al cursor que devuelve el método **`find`**(), utilizando una sintaxis de encadenamiento de puntos. Cada modificador se aplica invocándolo como un método del cursor con sus argumentos correspondientes. La sintaxis general para aplicar modificadores es la siguiente:

```
db.collection.find(query).modificador1(argumento1).modificador2(argumento2)...
```

Donde:

- **`db.collection.find(query)`**: Es la consulta inicial que devuelve un cursor.
- **`.modificador1(argumento1)`**: Es el primer modificador aplicado al cursor, con su argumento respectivo.
- **`.modificador2(argumento2`**: Es el segundo modificador aplicado, con su argumento respectivo.
- y así sucesivamente.

La combinación de modificadores proporciona una gran capacidad para precisar y optimizar las consultas, permitiendo un manejo eficiente de los datos, especialmente cuando se trabaja con grandes volúmenes de información o se requiere paginación en interfaces de usuario.

### `sort()`

El modificador **`sort()`** se usa para ordenar los documentos en el conjunto de resultados de una consulta **`find().`** Se puede especificar un campo por el que ordenar y la dirección de la ordenación: **`1`** para un orden ascendente y **`-1`** para un orden descendente.

Por ejemplo, si se desea obtener un listado de todos los documentos de la colección **`libros`** ordenado de forma ascendente por el campo **`año`**, se podría utilizar:

```
db.libros.find().sort({ año: 1 })
```

Y si se quiere obtener un listado de todos los documentos de la colección **`libros`** ordenado de forma descendente por el campo **`título`**, se podría utilizar:

```
db.libros.find().sort({ titulo: -1 })
```

### `limit()`

El modificador **`limit()`** se usa para limitar la cantidad de documentos que se devuelven en una consulta **`find().`** Si se establece en **`0,`** no hay límite en la cantidad de documentos devueltos.

Por ejemplo, si que quieren obtener únicamente cinco documentos de la colección **libros**, se utilizaría el comando:

```
db.libros.find().limit(5)
```

Si se quieren obtener los diez primeros documentos de la colección **libros** ordenados por **título** de forma ascendente, se usaría el comando:

```
db.libros.find().sort({ titulo: 1 }).limit(10)
```

### skip()

Este modificador se utiliza para saltarse una cantidad específica de documentos antes de empezar a devolver los resultados. Es útil para implementar la paginación de resultados en una aplicación.

Por ejemplo, generar un listado de todos los documentos de la colección **libros**, ignorando o saltando los diez primeros, se puede hacer con el comando:

```
db.libros.find().skip(10)
```

# Agregaciones

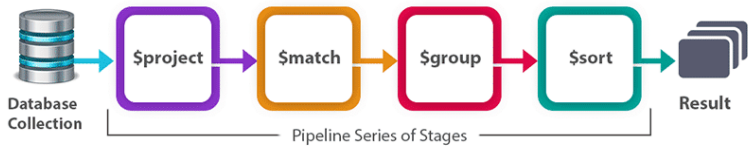
## Introducción

Las agregaciones en MongoDB son una forma poderosa y flexible de procesar datos y transformarlos en información útil. Permiten realizar operaciones complejas de procesamiento de datos directamente en la base de datos, lo que puede mejorar significativamente la eficiencia y la velocidad de las consultas y análisis de datos. La funcionalidad de agregación de MongoDB es similar en concepto a las operaciones de agrupamiento y join en las bases de datos SQL, pero ofrece una mayor flexibilidad y capacidad de procesamiento.

### ¿Qué son las agregaciones?

Una agregación es un proceso que pasa los datos de una colección a través de una serie de etapas, cada una de las cuales transforma los datos de alguna manera. Las agregaciones pueden simplificar los datos para el análisis, agrupar datos, calcular totales o promedios, filtrar datos, y mucho más.

### Etapas de una agregación




*Aggregation Pipeline* es el concepto central de las agregaciones en MongoDB. Imagina una tubería por la cual pasan los documentos, transformándose en cada etapa antes de llegar al final. Cada etapa de tubería transforma los documentos de alguna manera antes de pasarlos a la siguiente etapa. Algunas de las etapas más comunes son:

- **`$match`**: Filtra los documentos dejando pasar a la siguiente etapa solo aquellos que coinciden con las condiciones dadas. Es como un find() en una consulta regular.
- $**`group:`** Agrupa los documentos por algún identificador único y permite realizar operaciones de agregación, como contar, sumar, o calcular promedios, en cada grupo.
- **`$sort`**: Ordena los documentos.
- **`$project`**: Selecciona, excluye o crea campos de los documentos.
- **`$limit`**: Limita el número de documentos a pasar a la siguiente etapa.
- **`$skip`**: Salta un número específico de documentos.
- **`$unwind`**: Descompone un campo de tipo array de los documentos de entrada y produce un documento por cada elemento del array.

Como se ve en la lista anterior, cada etapa puede realizar acciones específicas sobre los documentos que le llegan:

- Filtrado y selección: Puedes decidir exactamente qué documentos o partes de documentos quieres que pasen a través de la tubería.
- Agrupación: Puedes agrupar documentos por uno o más campos y aplicar operaciones de agregación para resumir datos.
- Ordenación y limitación: Puedes ordenar los resultados de acuerdo a cualquier campo y limitar cuántos documentos quieres recibir.
- Transformación: Los documentos pueden ser transformados, añadiendo nuevos campos, eliminando campos existentes, o calculando nuevos valores.

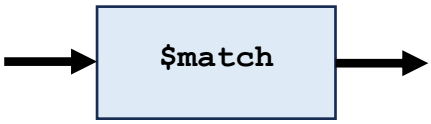
## `find()` o `aggregate()`

En MongoDB, tanto **`find()`** como **`aggregate()`** son comandos utilizados para recuperar y manipular datos, pero cada uno tiene propósitos y capacidades específicas. Su elección depende de la complejidad de la consulta y de las operaciones de transformación de datos que se necesite realizar.

- **`find()`** es el método más directo y comúnmente utilizado para recuperar documentos de una colección en MongoDB. Es ideal para consultas simples, donde necesitas seleccionar documentos que coinciden con ciertos criterios y posiblemente proyectar (es decir, incluir o excluir) ciertos campos de los documentos resultantes.
- **`aggregate()`** es mucho más potente y flexible, permitiendo realizar operaciones complejas de procesamiento de datos. Esto incluye filtrado, agrupación, ordenación, y muchas otras transformaciones de documentos en una serie de etapas, conocidas como pipeline de agregación. aggregate() es esencial cuando necesitas realizar operaciones complejas como agrupaciones, cálculos de sumatorias, promedios, o cuando deseas transformaciones de los datos que van más allá de una simple selección y proyección.

| | **find()** | **aggregate()** |
|---|---|---|
| **Propósito** | Consultas simples de selección de documentos. | Operaciones complejas de procesamiento de datos y transformaciones. |
| **Flexibilidad** | Menor. Solo permite filtrado básico y proyección. | Mayor. Permite múltiples operaciones como filtrado, proyección, agrupación, ordenación, etc. |
| **Rendimiento** | Generalmente más rápido para consultas simples. | Puede ser más lento dependiendo de la complejidad del *pipeline* de agregación. |
| **Complejidad de Consultas** | Baja. Ideal para consultas directas. | Alta. Soporta consultas y transformaciones complejas. |
| **Operaciones de Agrupación** | No soportadas directamente. | Soportadas, con capacidades extensas. |
| **Uso de Índices** | Soportado, lo que puede mejorar la velocidad de las consultas. | Soportado, pero la eficiencia puede variar según las etapas del *pipeline*. |

## Etapa `$match`




La etapa **`$match`** en las agregaciones de MongoDB se utiliza para filtrar los documentos de una colección, permitiendo que solo aquellos documentos que cumplan con los criterios especificados pasen a las siguientes etapas del *pipeline* de agregación. Funciona de manera similar al comando **`find()`**, pero dentro del contexto de una agregación, ofreciendo una forma potente y flexible de realizar filtrados iniciales o intermedios.

### Propósito

El propósito principal de $match es reducir el número de documentos sobre los cuales se realizarán operaciones en las siguientes etapas del *pipeline*. Esto puede mejorar significativamente el rendimiento de las agregaciones al disminuir la cantidad de datos que deben ser procesados en etapas subsiguientes.

**`$match`** es especialmente útil para:

- Excluir documentos que no son relevantes para el análisis o resultado deseado.
- Preparar el conjunto de datos para agregaciones más complejas, asegurando que solo los documentos de interés sean considerados.

### Sintaxis

La sintaxis de la etapa **`$match`** sigue el formato general de los operadores de consulta de MongoDB. Se especifica como un documento JSON, donde el campo representa el criterio de filtrado y el valor especifica las condiciones que deben cumplir los documentos:

```
{
  $match: {
    <campo>: <valor>,
    ...
  }
}
```

- **<campo>**: El nombre del campo por el cual se quieres filtrar.
- **<valor>**: El criterio de filtrado aplicado al campo. Puedes utilizar operadores de comparación (como **`$eq`**, **`$gt`**, **`$lt`**, etc.) para especificar condiciones más complejas.

### Ejemplo

Considera una colección ventas con documentos que representan ventas individuales. Si quieres filtrar solo aquellas ventas que se realizaron en el año 2023, la etapa $match de tu agregación se vería así:

```
db.ventas.aggregate([
  {
    $match: {
      "fechaVenta": {
        $gte: ISODate("2023-01-01"),
```

```
      $lte: ISODate("2023-12-31")
    }
  }
 }
 // Puedes añadir más etapas aquí...
])
```

Este ejemplo utiliza **`$match`** al principio del *pipeline* de agregación para filtrar los documentos. Sin embargo, **`$match`** también puede ser utilizado en cualquier parte del *pipeline*, permitiendo filtrados dinámicos basados en transformaciones de datos que ocurran en etapas anteriores.

## Etapa `$group`




La etapa **`$group`** en las agregaciones de MongoDB se utiliza para agrupar documentos de una colección según uno o más campos especificados y realizar operaciones de agregación sobre cada grupo de documentos. Esta etapa es crucial para sintetizar y resumir los datos, permitiendo obtener estadísticas y métricas consolidadas a partir de un conjunto de datos.

Propósito

El propósito de **`$group`** es reunir documentos que comparten un valor común en uno o varios campos y luego aplicar operaciones de agregación, como sumas, promedios, conteos, y más, a cada grupo. Esto es útil para:

- Calcular totales y promedios.
- Contar documentos en categorías específicas.
- Extraer estadísticas resumidas de grandes volúmenes de datos.

### Sintaxis

La sintaxis de la etapa **`$group`** requiere especificar un **`_id`** que define cómo se agruparán los documentos. Este **`_id`** puede ser un campo existente en los documentos o una expresión que resulte en un valor por el cual agrupar. Además, se pueden especificar campos adicionales para realizar operaciones de agregación:

```
{
 $group: {
  _id: <expresión de agrupación>,
  <campo1>: { <operación de agregación1>: <expresión> },
  ...
 }
}
```

- **<expresión de agrupación>**: Define cómo se agruparán los documentos. Puede ser un campo o una expresión.
- **<campo1>, <campo2>, ...**: Campos adicionales en el documento de resultado que contienen los resultados de las operaciones de agregación aplicadas al grupo.

### Ejemplo

Considerando nuevamente la colección ventas, supongamos que quieres calcular el total de ventas y el promedio de venta por producto para el año 2023. La etapa $group en tu agregación podría ser:

```
db.ventas.aggregate([
      // Etapa 1: ventas del año 2023
      {
    $match: {
     "fechaVenta": {
       $gte: ISODate("2023-01-01"),
       $lte: ISODate("2023-12-31")
     }
    }
  },
      // Etapa 2: agrupa por productoId y calcula total y promedio
  {
    $group: {
     _id: "$productoId",
     totalVentas: { $sum: "$monto" },
     promedioVenta: { $avg: "$monto" }
    }
  }
])
```

En este ejemplo:

- Etapa 1: Se filtran las ventas del año 2023 usando **`$match`**.
- Etapa 2: Se agrupan las ventas por **`productoId`** usando **`$group`**, calculando el total (**`$sum`**) y el promedio (**`$avg`**) de las ventas **`($monto)`** por producto.

Esta agregación proporcionaría una visión consolidada de las ventas por producto, permitiendo a las empresas identificar qué productos generaron mayores ingresos y cuál fue el valor de venta promedio, información crucial para la toma de decisiones estratégicas.

## Operaciones de agregación

La etapa **`$group`** en MongoDB es utilizada para agrupar documentos según un campo o expresión común y para realizar operaciones de agregación sobre esos grupos. Estas operaciones permiten calcular valores como sumas, promedios, mínimos, máximos, entre otros, para cada grupo de documentos. Aquí se describen las operaciones de agregación más comunes que puedes emplear en **`$group`**:

### $sum

- Suma los valores de un campo específico en cada grupo.
- Sintaxis: **`{ $sum: <expresión> }`**

Ejemplo: Para sumar el total de ventas por producto:

```
{
  $group: {
    _id: "$productoId",
    totalVentas: { $sum: "$monto" }
  }
}
```

### $avg

- Calcula el promedio de los valores de un campo en cada grupo.
- Sintaxis: **`{ $avg: <expresión> }`**

Ejemplo: Para obtener el promedio del monto de ventas por producto:

```
{
  $group: {
    _id: "$productoId",
    promedioVentas: { $avg: "$monto" }
  }
}
```

### $min

- Encuentra el valor mínimo de un campo en cada grupo.
- Sintaxis: **`{ $min: <expresión> }`**

Ejemplo: Para obtener la fecha de la venta más antigua por cliente:

```
{
  $group: {
    _id: "$cliente",
    fechaMinima: { $min: "$fechaVenta" }
  }
}
```

### $max

- Encuentra el valor máximo de un campo en cada grupo.
- Sintaxis: { **`$max: <expresión> }`**

Ejemplo: Para obtener el monto de la venta más alta por producto:

```
{
  $group: {
    _id: "$productoId",
    ventaMaxima: { $max: "$monto" }
  }
}
```

### $first

- Devuelve el primer documento del grupo. Generalmente, se usa junto con $sort para definir el orden.
- Sintaxis: **`{ $first: <expresión> }`**

**Ejemplo**: Para obtener el primer producto vendido por cada cliente:

```
{
  $group: {
    _id: "$cliente",
    primerProducto: { $first: "$productoId" }
  }
}
```

### $last

- Devuelve el último documento del grupo. Similar a $first, pero devuelve el último documento después de ordenar.
- Sintaxis: **`{ $last: <expresión> }`**

**Ejemplo**: Para obtener el último producto vendido por cada cliente:

```
{
  $group: {
    _id: "$cliente",
    ultimoProducto: { $last: "$productoId" }
  }
}
```

### $push

- Empuja todos los valores de un campo en cada grupo a un arreglo.
- Sintaxis: **`{ $push: <expresión> }`**

**Ejemplo**: Para agrupar todos los productos vendidos por cliente:

```
{
  $group: {
    _id: "$cliente",
    productosVendidos: { $push: "$productoId" }
  }
}
```

### $addToSet

- Similar a $push, pero solo agrega valores únicos a un arreglo.
- Sintaxis: **`{ $addToSet: <expresión> }`**

**Ejemplo**: Para obtener un conjunto único de productos vendidos por cliente:

```
{
  $group: {
    _id: "$cliente",
    productosUnicos: { $addToSet: "$productoId" }
```

```
 }
}
```

Estas son algunas de las operaciones más comunes que puedes emplear en la etapa **$group**. Puedes combinar diferentes operaciones de agregación dentro de una misma etapa **$group** para obtener resultados más complejos y detallados.

## Etapa $sort




La etapa **$sort** en las agregaciones de MongoDB se utiliza para ordenar los documentos procesados por la pipeline de agregación según uno o más campos. Los documentos pueden ser ordenados tanto en orden ascendente como descendente.

### Propósito

El propósito principal de **$sort** es organizar los documentos resultantes de una manera específica, lo que facilita su análisis o mejora la legibilidad de los resultados. Algunos usos comunes de **$sort** incluyen:

- Preparar los datos para una presentación o reporte ordenado.
- Ordenar los documentos para operaciones subsiguientes que dependan del orden, como limitar los resultados a los primeros n documentos.

### Sintaxis

La sintaxis de **$sort** especifica los campos por los cuales se deben ordenar los documentos y la dirección de la ordenación (**1** para ascendente y **-1** para descendente):

```
{
 $sort: {
  <campo1>: <dirección>,
  <campo2>: <dirección>,
  ...
 }
}
```

- **<campo1>, <campo2>,** ...: Los campos por los cuales se ordenarán los documentos.
- **<dirección>**: **1** para orden ascendente y **-1** para orden descendente.

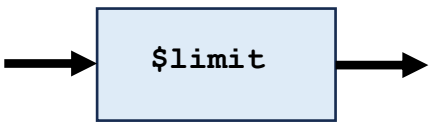
### Ejemplo

Suponiendo que deseas obtener un informe de las ventas totales por producto para el año 2023, pero ordenadas por el total de ventas descendente para identificar rápidamente los productos más vendidos, tu pipeline de agregación incluiría **$sort** de la siguiente manera:

```
db.ventas.aggregate([
  // Etapa 1: ventas del año 2023
  {
    $match: {
      "fechaVenta": {
        $gte: ISODate("2023-01-01"),
        $lte: ISODate("2023-12-31")
      }
    }
  },
  // Etapa 2: agrupa por productoId y calcula total
  {
    $group: {
      _id: "$productoId",
      totalVentas: { $sum: "$monto" }
    }
  },
  // Etapa 3: ordena por totalVentas en orden descendente
  {
    $sort: {
      totalVentas: -1
    }
  }
])
```

En este ejemplo, después de filtrar las ventas por el año 2023 y agruparlas por **`productoId`** para sumar el monto de las ventas, los documentos resultantes se ordenan en función del campo **`totalVentas`** en orden descendente. Esto significa que los primeros documentos en los resultados serán aquellos con mayores ventas totales, facilitando la identificación de los productos más exitosos.

## Etapa `$limit`




La etapa **`$`** en las agregaciones de MongoDB se utiliza para restringir el número de documentos que pasan a las etapas siguientes o que se devuelven como resultado final. Esto puede ser útil para limitar la salida de una agregación, especialmente cuando se espera un gran volumen de resultados.

### Propósito

El propósito principal de **`$limit`** es reducir el tamaño del resultado de una agregación a un número específico de documentos. Es especialmente útil cuando deseas obtener solo un subconjunto de resultados, como los primeros **`n`** documentos o un número fijo de resultados para pruebas o análisis rápidos. Algunos usos comunes de **`$limit`** incluyen:

- Limitar el número de resultados devueltos para mejorar la eficiencia.
- Obtener solo una muestra representativa de documentos.

- Controlar la cantidad de datos devueltos al cliente o al front-end para evitar desbordamientos o retrasos.

### Sintaxis

La sintaxis de **`$limit`** es sencilla: especifica el número máximo de documentos a devolver como resultado de la etapa:

```
{
  $limit: <número>
}
```

- **<número>**: El número máximo de documentos a permitir pasar a la siguiente etapa o a devolver como resultado final.

### Ejemplo

Supongamos que tienes una colección **ventas** y ya has filtrado y ordenado las ventas totales por producto. Ahora quieres obtener solo los 5 productos con mayores ventas. Para hacerlo, puedes utilizar **`$limit`** para obtener solo los primeros 5 documentos después de ordenar:

```
db.ventas.aggregate([
  // Etapa 1: ventas del año 2023
  {
    $match: {
      "fechaVenta": {
        $gte: ISODate("2023-01-01"),
        $lte: ISODate("2023-12-31")
      }
    }
  },
  // Etapa 2: agrupa por productoId y calcula total
  {
    $group: {
      _id: "$productoId",
      totalVentas: { $sum: "$monto" }
    }
  },
  // Etapa 3: ordena por totalVentas en orden descendente
  {
    $sort: {
      totalVentas: -1
    }
  },
  // Etapa 4: limita a 5 el número de documentos
  {
    $limit: 5
  }
])
```

En este ejemplo, el uso de **`$limit`** después de **`$sort`** garantiza que solo los 5 productos con mayores ventas en el año 2023 sean devueltos como resultado final. Esto es útil para

generar informes o análisis enfocados solo en los mejores resultados, mejorando la eficiencia y la claridad de los datos obtenidos.

## Etapa `$project`




La etapa **`$project`** en las agregaciones de MongoDB se utiliza para seleccionar, excluir, o transformar campos de los documentos en la pipeline de agregación. Esto permite refinar la estructura de los documentos que se pasan a las siguientes etapas o que se devuelven como resultado final.

### Propósito

El propósito de **`$project`** es controlar la forma y el contenido de los documentos resultantes, proporcionando la capacidad de:

- Incluir solo los campos necesarios en los resultados finales.
- Excluir campos innecesarios para reducir el tamaño de los resultados.
- Transformar o derivar nuevos campos a partir de los existentes.
- Renombrar campos para mejorar la claridad.

### Sintaxis

La sintaxis de **`$project`** permite definir qué campos deben ser incluidos o excluidos, así como crear campos nuevos o modificados:

```
{
   $project: {
     <campo1>: <inclusión/exclusión o transformación>,
     ...
   }
}
```

- **<campo1>, <campo2>, ...**: Los campos a incluir, excluir, o transformar.
- Para incluir un campo, se establece a **1**.
- Para excluir un campo, se establece a **0**.
- Para transformar o derivar un campo, se especifica una expresión que defina cómo se calcula el valor del nuevo campo.

### Ejemplo

Supongamos que estás trabajando con la colección **ventas** y deseas obtener información específica sobre las ventas para el año 2023. Sin embargo, solo te interesan los campos **productoId**, **totalVentas**, y quieres agregar un campo que calcule un porcentaje del total de ventas respecto a un objetivo de ventas para cada producto:

```
db.ventas.aggregate([
```

```
// Etapa 1: ventas del año 2023
{
  $match: {
    "fechaVenta": {
      $gte: ISODate("2023-01-01"),
      $lte: ISODate("2023-12-31")
    }
  }
},
// Etapa 2: agrupa por productoId y calcula total y promedio
{
  $group: {
    _id: "$productoId",
    totalVentas: { $sum: "$monto" }
  }
},
// Etapa 3: campos a mostrar (productoId,totalVentas) y campo
//          calculado (porcentajeObjetivo)
{
  $project: {
    productoId: 1,
    totalVentas: 1,
    porcentajeObjetivo: {
      $multiply: [ { $divide: ["$totalVentas", 1000000] }, 100]
    }
  }
 }
])
```

En este ejemplo, el uso de **`$project`** después de **`$group`** permite refinar el resultado para incluir solo los campos que te interesan. También se añade un campo p**`orcentajeObjetivo`** que calcula el porcentaje de ventas respecto a un objetivo (por ejemplo, 1,000,000). Esto muestra la flexibilidad de **`$project`** para seleccionar, excluir y transformar datos, proporcionando un nivel de control preciso sobre los resultados finales de un pipeline de agregación.

## Etapa `$count`




La etapa **`$count`** en las agregaciones de MongoDB se utiliza para contar el número de documentos que pasan a través del pipeline de agregación. Puede ser utilizada para obtener el conteo total de documentos que cumplen ciertas condiciones o para verificar la cantidad de documentos resultantes después de otras etapas de agregación.

### Propósito

El propósito de $count es proporcionar un conteo exacto de documentos. Esto puede ser útil para:

- Obtener un total de documentos que coinciden con ciertos criterios.

- Determinar la cantidad de resultados generados por una pipeline de agregación.
- Calcular métricas derivadas basadas en el número de documentos.

### Sintaxis

La sintaxis de $count es simple: especifica un nombre para el campo que contendrá el conteo total de documentos:

```
{
  $count: "<nombre_del_campo>"
}
```

- **<nombre_del_campo>:** El nombre del campo en el que se almacenará el conteo.

### Ejemplo

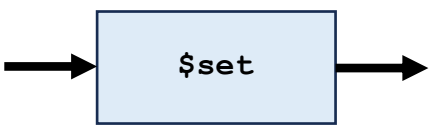
Supongamos que tienes una colección ventas y quieres contar cuántas ventas se realizaron en el año 2023 para entender el volumen total de ventas en ese período. Puedes usar **`$count`** después de un **`$match`** para obtener este conteo:

```
db.ventas.aggregate([
  // Etapa 1: ventas del año 2023
  {
    $match: {
      "fechaVenta": {
        $gte: ISODate("2023-01-01"),
        $lte: ISODate("2023-12-31")
      }
    }
  },
  // Etapa 2: contar documentos y almacenar valor en totalVentas
  {

    $count: "totalVentas"
  }
])
```

En este ejemplo, el pipeline de agregación primero filtra las ventas por el año 2023 usando **`$match`**. Luego, la etapa **`$count`** cuenta el número total de ventas y almacena el resultado en un campo llamado **`totalVentas`**. Esto es útil para obtener una cifra clara y precisa de cuántas ventas se realizaron en un período específico, lo que puede ser importante para análisis o informes.

## Etapa `$set`




La etapa **`$set`** en las agregaciones de MongoDB se utiliza para agregar nuevos campos a los documentos o para modificar los campos existentes en la pipeline de agregación. Esta etapa es versátil y te permite crear campos adicionales basados en cálculos, constantes o transformaciones de otros campos.

### Propósito

El propósito de **`$set`** es proporcionar una forma de enriquecer o modificar los documentos en la pipeline de agregación. Algunos usos comunes incluyen:

- Añadir campos calculados o derivados.
- Modificar valores existentes en documentos.
- Incluir información adicional basada en condiciones o relaciones con otros campos.

### Sintaxis

La sintaxis de **`$set`** es relativamente simple: especificas los campos que quieres agregar o modificar y la expresión o valor que determina el nuevo contenido del campo:

```
{
  $set: {
    <nombre_del_campo1>: <expresión_o_valor>,
    ...
}
}
```

- **<nombre_del_campo1>, <nombre_del_campo2>, ...:** Los campos que se agregarán o modificarán.
- **<expresión_o_valor>:** El valor o expresión que define el contenido del campo. Puede ser un valor fijo, una referencia a otro campo, o una expresión más compleja.

### Ejemplo

Imagina que tienes una colección ventas y quieres agregar un campo que represente el impuesto aplicado a cada venta, calculándolo como un porcentaje del monto de la venta. Puedes usar **`$set`** para agregar este campo a cada documento:

```
db.ventas.aggregate([
  // Etapa 1: ventas del año 2023
  {
    $match: {
      "fechaVenta": {
        $gte: ISODate("2023-01-01"),
        $lte: ISODate("2023-12-31")
      }
    }
  },
  // Etapa 2: añade campo impuesto a cada documento
  {
    $set: {
      impuesto: {
        $multiply: ["$monto", 0.1]
      }
    }
```

```
  }
])
```

En este ejemplo, después de filtrar las ventas por el año 2023, la etapa **`$set`** agrega un campo impuesto a cada documento, calculándolo como el 10% del campo monto. Esto muestra cómo **`$set`** puede usarse para enriquecer los documentos con información adicional o derivada, lo que puede ser útil para análisis, informes o procesamiento adicional.

## Etapa `$skip`




La etapa **`$skip`** en las agregaciones de MongoDB se utiliza para omitir un número específico de documentos al principio del pipeline de agregación o en cualquier etapa posterior. Esto es útil para descartar documentos que no se desean procesar o para implementar paginación y desplazamiento de resultados.

### Propósito

El propósito principal de **`$skip`** es permitir que una pipeline de agregación ignore un número determinado de documentos antes de continuar con las etapas subsiguientes. Algunas situaciones donde puede ser útil incluyen

- Implementar paginación para dividir resultados en partes más pequeñas.
- Descartar documentos irrelevantes o para procesamiento específico.
- Saltar documentos para obtener subconjuntos de datos.

### Sintaxis

La sintaxis de **`$skip`** es simple y consiste en especificar el número de documentos que deben ser omitidos:

```
{
  $skip: <número>
}
```

- **<número>:** El número de documentos que deseas omitir o saltar.

### Ejemplo

Considera que tienes una colección **`ventas`** y quieres obtener los resultados de ventas del año 2023, pero solo a partir del sexto documento, por ejemplo, para implementar una paginación. Puedes usar **`$skip`** para omitir los primeros cinco documentos:

```
db.ventas.aggregate([
  // Etapa 1: ventas del año 2023
  {
    $match: {
```

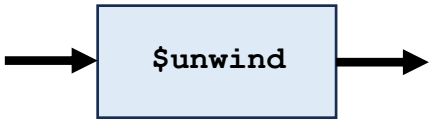
```
    "fechaVenta": {
      $gte: ISODate("2023-01-01"),
      $lte: ISODate("2023-12-31")
    }
  }
},
// Etapa 2: ordenar por fecha
{
  $sort: {
    "fechaVenta": 1
  }
},
// Etapa 3: ignorar los cinco primeros documentos
{
  $skip: 5
}
])
```

En este ejemplo:

- Se filtran las ventas por el año 2023 utilizando $match.
- Luego, se ordenan por la fecha de venta en orden ascendente usando $sort.
- Posteriormente, se usa **`$skip`** para omitir los primeros cinco documentos, de modo que el resultado final contiene solo los documentos a partir del sexto.

Esta implementación con **`$skip`** puede ser útil para paginación o para dividir resultados en partes más pequeñas, facilitando el procesamiento y análisis de grandes volúmenes de datos. Además, permite omitir documentos que no son relevantes para el análisis final, reduciendo el ruido en los resultados.

## Etapa `$unwind`




La etapa **`$unwind`** en las agregaciones de MongoDB se utiliza para descomponer documentos que contienen campos de tipo array, generando un nuevo documento por cada elemento del array. Esto es útil cuando se desea trabajar con cada elemento individualmente, permitiendo que las siguientes etapas de la pipeline traten a cada elemento como un documento separado.

### Propósito

El propósito de **`$unwind`** es descomponer documentos que contienen arreglos para obtener un documento separado por cada elemento del arreglo. Algunos usos comunes incluyen:

- Descomponer documentos para trabajar con elementos individuales.
- Transformar un arreglo en un conjunto de documentos para facilitar el análisis.
- Realizar operaciones de agregación en elementos individuales de un arreglo.

Sintaxis

La sintaxis de **`$unwind`** especifica el campo del documento que se debe descomponer:

```
{
  $unwind: {
    path: "<campo_del_arreglo>",
    preserveNullAndEmptyArrays: <true/false>
  }
}
```

- **`path`**: El nombre del campo que contiene el arreglo a descomponer.
- **`preserveNullAndEmptyArrays`** (opcional): Si es true, mantiene documentos que tienen campos nulos o arreglos vacíos; si es false, esos documentos se descartan. El valor predeterminado es false.

### Ejemplo 1:

Considera una colección ventas donde cada documento representa una venta y contiene un campo productos, que es un arreglo con todos los productos vendidos en esa transacción. Supongamos que deseas descomponer esta información para trabajar con cada producto vendido individualmente:

```
db.ventas.aggregate([
  // Etapa 1: ventas del año 2023
  {
    $match: {
      "fechaVenta": {
        $gte: ISODate("2023-01-01"),
        $lte: ISODate("2023-12-31")
      }
    }
  },
  // Etapa 2: descomponer el array productos
  {
    $unwind: {
      path: "$productos",
      preserveNullAndEmptyArrays: false
    }
  }
])
```

En este ejemplo:

- Se filtran las ventas para el año 2023 utilizando **`$match`**.
- Luego, se usa **`$unwind`** para descomponer el campo **`productos`**, generando un documento separado por cada producto en el array. Si el campo está vacío o es nulo, el documento se descarta porque **`preserveNullAndEmptyArrays`** se establece en **`false`**.

Esta aplicación de **`$unwind`** permite trabajar con cada producto vendido en 2023 como un documento individual, facilitando operaciones adicionales como agrupaciones,

conteos o cálculos específicos sobre los productos vendidos. También es útil cuando necesitas descomponer datos para obtener detalles más granulares o para operaciones que requieren cada elemento de un arreglo como documento

### Ejemplo 2:

Un ejemplo puede aclarar cómo funciona la etapa **`$unwind`** en un pipeline de agregación en MongoDB. Consideremos un documento en una colección llamada **`ventas`** que contiene un campo de tipo arreglo llamado **`productos`**, que enumera los productos vendidos en una transacción. Aquí está el documento original:

```
{
  "_id": 1,
  "fechaVenta": ISODate("2023-02-15"),
  "cliente": "Juan Pérez",
  "productos": [
    {"productoId": "p1", "nombre": "Producto A", "cantidad": 2},
    {"productoId": "p2", "nombre": "Producto B", "cantidad": 1}
  ],
  "total": 50.00
}
```

En este documento, el campo **`productos`** es un array que contiene información sobre dos productos vendidos en una transacción. Ahora vamos a aplicar **`$unwind`** para descomponer este arreglo, generando un documento por cada elemento del arreglo.

Pipeline con **`$unwind`**

```
db.ventas.aggregate([
  {
    $unwind: {
      path: "$productos",
      preserveNullAndEmptyArrays: false
    }
  }
])
```

El resultado de este pipeline sería dos documentos separados, uno para cada producto en el array **`productos`**:

Primer Documento:

```
{
  "_id": 1,
  "fechaVenta": ISODate("2023-02-15"),
  "cliente": "Juan Pérez",
  "productos": {"productoId": "p1", "nombre": "Producto A", "cantidad": 2},
  "total": 50.00
}
```

Segundo documento:

```
{
  "_id": 1,
  "fechaVenta": ISODate("2023-02-15"),
  "cliente": "Juan Pérez",
  "productos": {"productoId": "p2", "nombre": "Producto B", "cantidad": 1},
  "total": 50.00
}
```

Como se puede ver, **`$unwind`** ha descompuesto el arreglo **`productos`**, generando dos documentos separados: uno para el primer producto y otro para el segundo producto en la lista. Esto permite trabajar con cada producto de forma individual, lo que puede ser útil para análisis, agrupaciones, cálculos, u otras operaciones que requieran tratar elementos del arreglo como documentos independientes.

Si el campo **`productos`** estuviera vacío o fuera nulo, el documento se eliminaría de la salida si se usa **`preserveNullAndEmptyArrays: false.`** Si fuera **`true`**, se mantendría el documento pero con el campo **`productos`** como nulo o vacío.

## Etapa `$lookup`




La etapa **`$lookup`** en las agregaciones de MongoDB se utiliza para realizar un join entre dos colecciones, permitiendo combinar datos de una colección con datos relacionados de otra. Esto es especialmente útil para enriquecer documentos de una colección con información adicional de otra colección, similar a un join en bases de datos relacionales.

### Propósito

El propósito de $lookup es facilitar la combinación de datos entre colecciones. Algunos usos comunes incluyen:

- Combinar datos relacionados entre diferentes colecciones.
- Agregar información adicional a documentos de una colección utilizando datos de otra colección.
- Realizar análisis más complejos basados en relaciones entre colecciones.

### Sintaxis

La sintaxis de $lookup requiere varios parámetros clave para especificar cómo se realiza el join entre las colecciones:

```
{
  $lookup: {
    from: "<nombre_colección_relacionada>",
    localField: "<campo_local>",
    foreignField: "<campo_relacionado>",
    as: "<nombre_del_campo_resultado>"
  }
}
```

- **`from`**: El nombre de la colección con la que deseas hacer el join.
- **`localField`**: El campo de la colección actual que se usa para emparejar con la colección relacionada.
- **`foreignField`**: El campo de la colección relacionada que se usa para emparejar con la colección actual.
- **`as`**: El nombre del campo que contendrá los datos combinados del join.

### Ejemplo

Imagina que tienes una colección **`ventas`** y otra colección **`clientes`**. La colección **`ventas`** contiene información sobre ventas y la colección **`clientes`** tiene detalles sobre clientes, como nombre y dirección. Supongamos que quieres obtener la información de los clientes para cada venta. Puedes usar **`$lookup`** para combinar datos entre estas dos colecciones:

```
db.ventas.aggregate([
 {
  $lookup: {
   from: "clientes",
   localField: "clienteId",
   foreignField: "clienteId",
   as: "detalleCliente"
  }
 }
])
```

En este ejemplo:

- **`from`** es "**`clientes`**", la colección que contiene información sobre los clientes.
- **`localField`** es "**`clienteId`**", el campo en ventas que identifica al cliente.
- **`foreignField`** es también "**`clienteId`**", el campo en clientes que coincide con localField.
- **`as`** es "**`detalleCliente`**", el campo donde se almacenarán los datos combinados.

Después de aplicar este pipeline de agregación, cada documento en **`ventas`** incluirá un campo "**`detalleCliente`**" que contiene un arreglo con los datos relacionados de la colección **`clientes`**. Esto permite acceder a información detallada sobre los clientes para cada venta, facilitando el análisis o informes complejos que requieren datos de múltiples colecciones.

## Etapa `$out`




La etapa **`$out`** en las agregaciones de MongoDB se utiliza para escribir los resultados finales de una pipeline de agregación en una nueva colección o reemplazar el contenido de una colección existente. A diferencia de otras etapas que transforman datos, **`$out`** es una etapa terminal que

redirige los resultados a una colección específica y, por tanto, debe colocarse en último lugar en la agregación.

### Propósito

El propósito de **`$out`** es guardar el resultado de una agregación en una colección para un uso posterior, lo que puede ser útil para:

- Crear subconjuntos de datos para informes o análisis adicionales.
- Almacenar resultados intermedios para optimizar procesos de análisis.
- Generar datos de salida para integraciones con otros sistemas o procesos.

### Sintaxis

La sintaxis de **`$out`** requiere especificar el nombre de la colección donde se guardarán los resultados:

```
{
  $out: "<nombre_de_la_colección>"
}
```

- <nombre_de_la_colección>: El nombre de la colección donde se almacenarán los resultados.

Si la colección especificada no existe, MongoDB la creará. Si existe, sus documentos serán reemplazados por los resultados del pipeline de agregación.

### Ejemplo

Supongamos que tienes una colección **`ventas`** y deseas crear una nueva colección que contenga solo las ventas del año 2023, pero agrupadas por producto con el total de ventas para cada producto. Puedes usar **`$ou`**t para escribir este resultado a una colección llamada **`ventas_2023`**:

```
db.ventas.aggregate([
  // Etapa 1: ventas del año 2023
  {
    $match: {
      "fechaVenta": {
        $gte: ISODate("2023-01-01"),
        $lte: ISODate("2023-12-31")
      }
    }
  },
  // Etapa 2: agrupa por productoId y calcula total
  {
    $group: {
      _id: "$productoId",
      totalVentas: { $sum: "$monto" }
    }
  },
```

```
  // Etapa 3: resultado almacenar en colección ventas_2023
  {
    $out: "ventas_2023"
  }
])
```

En este ejemplo, después de filtrar las ventas por el año 2023 y agruparlas por **`productoId`** para sumar el total de **`ventas`**, la etapa **`$out`** escribe estos resultados a una nueva colección llamada **`ventas_2023`**. Esto es útil para crear una colección separada con datos consolidados para análisis posteriores o para compartir resultados con otras partes del sistema sin afectar la colección original **`ventas`**.

### Advertencia

La etapa **`$out`** es destructiva cuando se usa en una colección existente, por lo que debes tener precaución al usarla para evitar sobrescribir datos importantes. Asegúrate de hacer copias de seguridad si es necesario antes de usar esta etapa en una colección existente.

## Uso de `$project` y `$set`

La etapa **`$project`** y la etapa **`$set`** en las agregaciones de MongoDB tienen algunas similitudes en su funcionalidad, pero se utilizan para propósitos ligeramente diferentes. Aquí está la comparación entre ambas y cuándo usar cada una:

### Similitudes

- Ambas etapas pueden modificar la estructura de los documentos en un pipeline de agregación.
- Se pueden usar para añadir, modificar o eliminar campos de un documento.
- Permiten derivar campos basados en cálculos u otras transformaciones.

### Diferencias

- **`$project`** es generalmente utilizado para seleccionar, excluir, renombrar o transformar campos. Se puede usar para determinar qué partes del documento se devuelven o cómo se estructuran.
- **`$set`** está más enfocado en agregar nuevos campos o modificar campos existentes sin necesariamente eliminar otros campos del documento. Con **`$set`**, puedes añadir información a un documento sin tocar su estructura general.

### Cuándo usar $project

Usa **`$project`** cuando necesites:

- Seleccionar campos específicos para incluir o excluir en el resultado final.
- Renombrar campos para mayor claridad.
- Aplicar transformaciones más complejas o crear campos derivados.

### Ejemplo

Si quieres seleccionar solo algunos campos y renombrar otros en la colección ventas, puedes usar **`$project`**:

```
db.ventas.aggregate([
 {
  $project: {
   nombreProducto: 1,
   precioVenta: 1,
   fechaVenta: 1,
   producto: "$nombreProducto",
   precio: "$precioVenta"
  }
 }
])
```

En este ejemplo, se seleccionan solo algunos campos del documento original y se renombran **`nombreProducto`** y **`precioVenta`** a **`producto`** y **`precio`**, respectivamente.

### Cuándo Usar $set

Usa $set cuando necesites:

- Agregar nuevos campos sin alterar la estructura existente.
- Modificar campos existentes sin cambiar otros campos.
- Añadir información derivada o calculada a documentos existentes.

### Ejemplo

Si deseas agregar un campo que calcule un impuesto del 10% sobre el monto de cada venta en la colección **`ventas`**, puedes usar **`$set`**:

```
db.ventas.aggregate([
 {
  $set: {
   impuesto: {
    $multiply: ["$monto", 0.1]
   }
  }
 }
])
```

En este ejemplo, se agrega un nuevo campo **`impuesto`** que es el resultado de multiplicar el campo **`monto`** por **`0.1`**, sin modificar la estructura general del documento.

### Conclusión

Ambas etapas son versátiles y se pueden usar para modificar documentos, pero **`$project`** es más adecuado para trabajar con la estructura y **`$set`** para agregar o cambiar campos sin eliminar otros. La elección entre ambas depende de tu necesidad

específica: simplificar y seleccionar campos (usa **`$project`**) o agregar y modificar campos sin alterar la estructura general (usa **`$set`**).

# Índices

## Introducción

El rendimiento de una base de datos no solo depende de su diseño y del volumen de datos almacenados, sino también de la eficiencia con la que se pueden recuperar los documentos. En MongoDB, los **índices** juegan un papel fundamental en la **optimización de las consultas**, permitiendo acceder a la información de manera **rápida y eficiente**.

Un **índice** es una **estructura de datos auxiliar** que almacena una **copia ordenada de ciertos valores clave** de una colección, facilitando la búsqueda de documentos sin necesidad de recorrer toda la base de datos. En **bases de datos muy grandes**, donde millones o incluso miles de millones de documentos pueden estar almacenados, el uso adecuado de los índices **marca una gran diferencia en los tiempos de respuesta**. Sin un índice, MongoDB debe realizar un **escaneo completo de la colección (collection scan)**, lo que puede ser **extremadamente costoso** en términos de rendimiento.

Los índices en MongoDB permiten que las consultas sean **más eficientes**, ya que **reducen la cantidad de documentos** que deben ser explorados para encontrar los resultados deseados. Al mejorar el acceso a los datos, los índices pueden **acelerar significativamente** operaciones como **búsquedas, ordenamientos y combinaciones de datos en agregaciones**. Sin embargo, su uso también implica ciertas consideraciones, como el **consumo adicional de almacenamiento** y el **impacto en el rendimiento de las escrituras** ya que cada vez que se inserta, actualiza o elimina un documento, MongoDB debe **actualizar los índices** asociados, lo que puede ralentizar estas operaciones, especialmente en colecciones con una gran cantidad de índices

Para crear un índice en MongoDB, se utiliza el método **`createIndex`**(), cuya sintaxis general es:

```
db.<collection>.createIndex({<campo>: <orden>, <options>...})
```

Donde:

- **`<collection>`** es el nombre de la colección sobre la cual se creará el índice.
- **`<campo>`** representa el campo sobre el que se indexará la información.
- <**`orden`**> puede ser 1 (ascendente) o -1 (descendente).
- **`<options>`** son opciones adicionales que permiten personalizar el índice, como **`unique`**, **`sparse`** o **`expireAfterSeconds`**, entre otras.

En este capítulo, exploraremos los **diferentes tipos de índices** disponibles en MongoDB, **cómo se crean y administran**, y las **mejores prácticas** para su implementación. También analizaremos cómo los índices pueden **transformar el rendimiento de las consultas**, haciendo que las bases de datos documentales sean **más eficientes y escalables**.

# Tipos de índices

MongoDB ofrece una variedad de tipos de índices diseñados para optimizar la búsqueda y recuperación de datos de manera eficiente. Cada tipo de índice tiene una funcionalidad específica y se adapta a diferentes necesidades según la estructura de los documentos y los patrones de consulta utilizados en la base de datos.

El uso adecuado de los índices puede marcar una gran diferencia en el rendimiento de las consultas, reduciendo la cantidad de documentos escaneados y acelerando la ejecución de operaciones. Sin embargo, es importante seleccionar el tipo de índice correcto según el caso de uso, ya que los índices consumen espacio adicional en disco y pueden afectar el rendimiento de las escrituras.

En este capítulo, exploraremos algunos de los índices más utilizados en MongoDB:

- **Índice en el campo _id**: Es el índice único que MongoDB crea automáticamente en cada colección para garantizar la identificación rápida de documentos.
- **Índice simple**: Se aplica a un solo campo y mejora el rendimiento de las consultas que filtran o ordenan por dicho campo.
- **Índice compuesto**: Permite indexar múltiples campos en un solo índice, optimizando consultas que utilizan más de un criterio de búsqueda.
- **Índice multikey**: Se usa para indexar arrays dentro de los documentos, permitiendo consultas eficientes sobre elementos de una lista.

A lo largo de esta sección, veremos en detalle cómo funcionan estos índices, cómo se crean y en qué escenarios son más efectivos. Comprender su aplicación y sus limitaciones permitirá mejorar la eficiencia de las consultas en bases de datos documentales.

## Índice en el campo _id

MongoDB crea automáticamente un **índice único en el campo _id** para cada documento en una colección.

Las búsquedas por el campo `_id` son **extremadamente eficientes** en MongoDB debido a que este campo cuenta con un **índice único y predefinido** en cada colección. Cuando se realiza una consulta utilizando `_id`, MongoDB no necesita recorrer todos los documentos de la colección (*collection scan*), sino que accede directamente al índice para localizar el documento en cuestión. Esto reduce drásticamente el tiempo de búsqueda, incluso en bases de datos con millones de registros. Debido a esta eficiencia, siempre que sea posible, se recomienda utilizar `_id` como criterio principal en las consultas, especialmente en operaciones que requieren un acceso rápido y directo a un documento específico.

### Características del índice en _id

- **Creación automática**: Todas las colecciones en MongoDB generan este índice de forma predeterminada, sin necesidad de que el usuario lo defina manualmente.
- **Clave única**: El campo _id debe ser único dentro de la colección, evitando duplicados en la base de datos.

- **Optimización en búsquedas**: Consultar documentos por el campo _id es la forma más rápida de recuperarlos, ya que MongoDB accede directamente al índice sin necesidad de escanear toda la colección.
- **No se puede eliminar**: Aunque MongoDB permite eliminar otros índices creados manualmente, el índice en _id es obligatorio y no puede ser eliminado.

### Ejemplo de uso

Al insertar un documento sin especificar el campo _id, MongoDB lo genera automáticamente utilizando un **ObjectId**, una estructura especial de 12 bytes.

```
db.usuarios.insertOne({
  nombre: "Carlos",
  edad: 30});
```

Este documento se almacenará con un _id generado automáticamente, similar a este:

```
{  "_id": ObjectId("65f08c5c123456789abcde12"),
  "nombre": "Carlos",
  "edad": 30 }
```

Si intentamos insertar un documento con el mismo _id, MongoDB generará un error, ya que la clave debe ser única:

```
db.usuarios.insertMany([
  { _id: 1, nombre: "Ana", edad: 25 },
  { _id: 1, nombre: "Luis", edad: 28 } // Error: Duplicated _id
]);
```

### Consideraciones adicionales

- Se pueden utilizar valores personalizados para _id, como cadenas o números, pero deben ser **únicos**.
- Si se requiere modificar el valor de _id, MongoDB no lo permite directamente. La única opción es eliminar el documento y volver a insertarlo con el nuevo _id.
- Aunque todas las colecciones tienen este índice, MongoDB permite crear **colecciones sin índice en _id**, pero solo en bases de datos específicas como **colecciones temporales o de tipo *capped*** (colecciones de tamaño fijo que almacenan datos en orden de inserción y eliminan automáticamente los documentos más antiguos cuando se alcanza el límite de espacio asignado).

## Índice simple

Los **índices simples** en MongoDB se aplican a un solo campo dentro de una colección y permiten optimizar consultas que filtran, ordenan o buscan documentos basándose en dicho campo. A diferencia del índice en _id, que se crea automáticamente, los índices simples deben ser definidos manualmente según las necesidades del sistema.

### Características de los índices simples

- Se aplican a un solo campo de la colección.

- Solo se debe especificar un <campo> al momento de crearlo.
- La dirección del índice se define mediante <orden>:
    - 1 para orden ascendente.
    - -1 para orden descendente.

### Creación de un índice simple

Para crear un índice en un campo específico, se utiliza el método createIndex().

```
db.usuarios.createIndex({ nombre: 1 });
```

Este comando genera un **índice ascendente** en el campo "nombre", permitiendo que las búsquedas y ordenamientos sobre este campo sean más eficientes.

Si queremos buscar todos los usuarios llamados “Ana”, MongoDB podrá aprovechar el índice para encontrar el documento rápidamente:

```
db.usuarios.find({ nombre: "Ana" });
```

Además, este índice también se utiliza cuando se ordenan los resultados por el campo "nombre":

```
db.usuarios.find().sort({ nombre: 1 });
```

Si en lugar de un índice ascendente queremos ordenar los resultados de forma descendente, podemos crearlo con -1:

```
db.usuarios.createIndex({ nombre: -1 });
```

Esto optimizará consultas donde se necesite recuperar datos en orden inverso, como obtener la última persona alfabéticamente en la base de datos:

```
db.usuarios.find().sort({ nombre: -1 });
```

### Consideraciones adicionales

- **Mejoran el rendimiento de las consultas** en campos específicos, evitando escaneos completos de la colección.
- **Facilitan la ordenación eficiente** de los resultados según el campo indexado.
- **Afectan las operaciones de escritura**, ya que cada nueva inserción o actualización debe mantener el índice actualizado.
- **Consumen almacenamiento adicional**, por lo que es recomendable solo indexar los campos utilizados con frecuencia en búsquedas u ordenamientos.

## Índice compuesto

Los **índices compuestos** en MongoDB permiten indexar **varios campos en un solo índice**, optimizando las consultas que utilizan múltiples criterios de búsqueda. A diferencia de

los índices simples, que afectan solo a un campo, los índices compuestos organizan los datos de acuerdo con **el orden en el que se definen los campos** dentro del índice.

Un aspecto fundamental de los índices compuestos es que **el primer campo definido tiene mayor relevancia**, ya que MongoDB organiza los datos en función de este campo antes de considerar los siguientes. Esto significa que las consultas que filtran o buscan por **el primer campo del índice** pueden aprovechar completamente la optimización del índice. En cambio, las búsquedas que utilizan solo los **campos secundarios del índice** sin incluir el primero **no se beneficiarán del índice de manera eficiente**.

Por ejemplo, si creamos un índice compuesto en { nombre: 1, edad: -1 }, MongoDB ordenará primero los documentos por **nombre** en orden ascendente y, dentro de los nombres iguales, los ordenará por **edad** en orden descendente. Esto permite optimizar consultas que filtran por nombre, o que combinan nombre con edad, pero no aquellas que buscan únicamente por edad, ya que el índice no está estructurado para priorizar ese campo.

### Características de los índices compuestos

- Se generan sobre **varios campos** de la colección.
- Cada campo en el índice tiene un **orden específico** (1 para ascendente, -1 para descendente).
- Los datos se **agrupan y ordenan** siguiendo el orden en que se definieron los campos dentro del índice.
- Son **útiles para consultar uno o varios de los campos**, sin necesidad de incluirlos todos en la búsqueda.

### Creación de un índice compuesto

Para definir un índice compuesto, se especifican múltiples campos dentro del método createIndex():

```
db.usuarios.createIndex({ nombre: 1, edad: -1 });
```

Este índice ordenará primero los documentos por **`nombre`** en orden ascendente y, dentro de los mismos nombres, ordenará por **`edad`** en orden descendente.

Si queremos buscar usuarios por nombre y edad, MongoDB podrá aprovechar el índice para acelerar la consulta:

```
db.usuarios.find({ nombre: "Ana", edad: 30 });
```

También podemos buscar solo por el **primer campo** del índice (nombre en este caso), y MongoDB seguirá utilizando el índice:

```
db.usuarios.find({ nombre: "Ana" });
```

Sin embargo, si intentamos buscar solo por edad, MongoDB **no podrá usar este índice de manera eficiente**, ya que el índice está estructurado para agrupar primero por nombre.

Si usamos un índice compuesto para ordenar los resultados, MongoDB podrá optimizar la operación:

```
db.usuarios.find().sort({ nombre: 1, edad: -1 });
```

Este ordenamiento se beneficiará del índice ya definido, haciendo que la consulta sea mucho más eficiente.

### Consideraciones sobre índices compuestos

- Son más eficientes que múltiples índices simples cuando una consulta usa varios criterios de búsqueda.
- Permiten búsquedas optimizadas sobre el primer campo del índice o combinaciones de los campos indexados.
- No se pueden utilizar eficientemente si se consulta solo por un campo que no sea el primero en la definición del índice.
- Afectan el rendimiento de escritura más que los índices simples, ya que cada actualización de datos requiere mantener la estructura de varios campos indexados.

## Índice multikey

El **índice multikey** en MongoDB se utiliza para indexar **campos que contienen arrays**, permitiendo realizar búsquedas eficientes sobre los elementos individuales dentro de esos arrays. A diferencia de los índices simples y compuestos, que operan sobre valores escalares, los índices multikey manejan múltiples valores por documento, lo que los hace ideales para consultas que buscan dentro de listas o conjuntos de datos almacenados en un solo campo.

### Características de los índices multikey

- Se crean automáticamente cuando se declara un **índice sobre un campo de tipo array**.
- **Cada elemento dentro del array es indexado de forma individual**, permitiendo realizar búsquedas eficientes.
- Soportan consultas que filtran por uno o más valores del array sin necesidad de recorrer todos los documentos de la colección.
- Se pueden combinar con índices compuestos, pero con ciertas restricciones en los campos adicionales.

### Creación de un índice multikey

Cuando se indexa un campo que almacena arrays, MongoDB crea automáticamente un índice multikey sin necesidad de especificarlo explícitamente.

```
db.productos.createIndex({ categorias: 1 });
```

Si los documentos en la colección tienen arrays en el campo **`categorias`**, como en el siguiente caso:

```
{ "_id": 1, "nombre": "Laptop", "categorias": ["Tecnología", "Electrónica"] }
{ "_id": 2, "nombre": "Auriculares", "categorias": ["Electrónica", "Audio"] }
```

El índice multikey permitirá buscar productos que pertenezcan a una categoría específica de manera eficiente:

```
db.productos.find({ categorias: "Electrónica" });
```

MongoDB utilizará el índice para encontrar todos los documentos que contengan "**`Electrónica`**" dentro del array **`categorias`**, sin necesidad de hacer un escaneo completo de la colección.

También podemos realizar búsquedas por **múltiples valores dentro del array**:

```
db.productos.find({ categorias: { $all: ["Tecnología", "Electrónica"] } });
```

Esta consulta devolverá los documentos que contengan **ambos valores** en el array **`categorias`**.

### Restricciones y consideraciones

- No se pueden crear índices multikey en más de un campo dentro del mismo índice compuesto.

```
// No permitido si ambos son arrays
db.productos.createIndex({ categorias: 1, etiquetas: 1 });
```

- **El índice ocupa más espacio en disco**, ya que almacena referencias para cada elemento del array.
- **Afectan el rendimiento de escritura**, ya que cada modificación en un array puede requerir actualizar múltiples entradas en el índice.

## Opciones de Índices

Además de definir el tipo de índice adecuado para cada caso, MongoDB permite configurar diversas **opciones** al momento de crear un índice. Estas opciones permiten personalizar el comportamiento del índice para optimizar consultas, restringir valores y mejorar el rendimiento en determinadas situaciones.

### `unique` - restricción de unicidad

Si se define un índice con la opción "**`unique": true`**, MongoDB **obliga a que los valores del campo indexado sean únicos** dentro de la colección. Si se intenta insertar un documento con un valor duplicado en dicho campo, se generará un error.

#### Ejemplo:

```
db.usuarios.createIndex({ email: 1 }, { unique: true });

db.usuarios.insertMany([
  { nombre: "Ana", email: "ana@email.com" },
  { nombre: "Carlos", email: "ana@email.com" } // ERROR: email duplicado
]);
```

En este ejemplo, se intenta insertar dos documentos en la colección usuarios, ambos con el mismo valor en el campo **`email`**. Sin embargo, dado que previamente se creó un **índice único** en el campo **`email ({unique: true}`**), MongoDB impide la inserción del segundo documento, generando un error por **valor duplicado**. Esto garantiza que no haya correos electrónicos repetidos en la base de datos, asegurando la integridad de los datos y evitando registros duplicados.

### `sparse`: Índice solo sobre documentos con el campo presente

Cuando se define un índice con la opción **`"sparse": true`**, **solo se indexan los documentos que contienen el campo especificado**, ignorando aquellos que no lo tienen. Esto ahorra espacio y mejora el rendimiento en colecciones con datos heterogéneos.

### Ejemplo:

```
db.usuarios.createIndex({ telefono: 1 }, { sparse: true });
```

Si hay documentos sin el campo **`telefono`**, no serán considerados en el índice, y las consultas que usen este índice solo se aplicarán a los documentos que sí tienen ese campo.

## `expireAfterSeconds`: Índice con tiempo de vida (TTL)

Este tipo de índice se usa comúnmente para **eliminar automáticamente documentos después de un tiempo determinado**, lo cual es útil en escenarios como sesiones temporales, cachés o registros de actividad que solo deben mantenerse por un período específico.

### Ejemplo:

Para establecer un índice con tiempo de vida en un campo de fecha, utilizamos el método createIndex() con la opción { expireAfterSeconds: <segundos> }.

```
db.sesiones.createIndex({ fechaExpiracion: 1 }, { expireAfterSeconds: 3600 });
```

Entonces:

- MongoDB crea un índice sobre el campo **`fechaExpiracion`**.
- Los documentos en la colección **`sesiones`** deben incluir el campo **`fechaExpiracion`** con un valor de tipo **`Date`**.
- 3600 segundos (1 hora) después de la fecha almacenada en **`fechaExpiracion`**, el documento será eliminado automáticamente de la base de datos.
- MongoDB **no elimina el índice**, solo los documentos cuya fecha de expiración haya pasado.

Si insertamos los siguientes documentos en la colección sesiones:

```
db.sesiones.insertMany([
  { usuario: "Ana", fechaExpiracion: new Date("2025-03-08T12:00:00Z") },
  { usuario: "Carlos", fechaExpiracion: new Date("2025-03-08T13:00:00Z") }
]);
```

Entonces:

- El documento de "Ana" se eliminará automáticamente cuando el reloj marque **13:00 UTC**, ya que habrán pasado 3600 segundos desde 12:00:00 UTC.
- El documento de "Carlos" se eliminará cuando el reloj marque **14:00 UTC**, siguiendo la misma lógica.

Esto evita que la colección crezca indefinidamente con datos obsoletos, sin necesidad de ejecutar eliminaciones manuales.

### Consideraciones importantes

- **Solo** funciona con campos de tipo Date. Si el campo fechaExpiracion tiene otro tipo de dato, MongoDB ignorará el índice TTL.

- MongoDB no elimina los documentos inmediatamente después de la expiración, sino que ejecuta un proceso en segundo plano cada ciertos minutos para revisar y borrar los documentos expirados.
- No se pueden utilizar índices TTL en índices compuestos, a menos que el campo con TTL sea el primero en el índice.
- El índice sigue existiendo después de la eliminación de los documentos. MongoDB solo borra los registros vencidos, pero el índice TTL permanece activo y se aplica a futuros documentos insertados.

Los **índices TTL** son una solución eficiente para la limpieza automática de datos temporales sin necesidad de tareas manuales o scripts adicionales.

## `partialFilterExpression`: Índice sobre documentos que cumplen una condición

Permite crear un **índice parcial**, es decir, un índice que **solo se aplica a los documentos que cumplen una condición específica**. Esto es útil cuando no queremos indexar toda la colección, sino solo un subconjunto de documentos que realmente serán utilizados en consultas.

Para definir un índice con **partialFilterExpression**, se debe especificar la condición dentro del índice:

```
db.usuarios.createIndex(
  { edad: 1 },
  { partialFilterExpression: { edad: { $gte: 18 } } }
);
```

En este caso:

- **`{ edad: 1 }`** → Se crea un índice **ascendente** en el campo **`edad`**.
- **`partialFilterExpression: { edad: { $gte: 18 } }`** → Solo los documentos donde **`edad`** sea **18 o mayor** se incluirán en el índice.
- **Documentos sin el campo edad o con valores menores a 18 no estarán indexados**, lo que ahorra espacio y mejora el rendimiento en consultas que buscan adultos.
- Si se realiza una búsqueda de usuarios adultos (edad >= 18), MongoDB **usará el índice** para acelerar la consulta. En cambio, si se buscan usuarios menores de edad (edad < 18), **el índice no se aplicará**, ya que esos documentos no fueron indexados.

Por ejemplo, si tenemos la siguiente colección usuarios:

```
{ "_id": 1, "nombre": "Ana", "edad": 25 }
{ "_id": 2, "nombre": "Carlos", "edad": 17 }
{ "_id": 3, "nombre": "Elena", "edad": 30 }
{ "_id": 4, "nombre": "Luis" }  // No tiene el campo "edad"
```

Con el índice parcial definido en { edad: 1 }, **solo los documentos de "Ana" y "Elena" estarán en el índice**, ya que cumplen con edad >= 18. Los documentos de "Carlos" y "Luis" no serán indexados.

Ahora, si hacemos la consulta:

```
db.usuarios.find({ edad: { $gte: 18 } });
```

MongoDB **utilizará el índice** para devolver los resultados rápidamente.

Sin embargo, si buscamos valores fuera del rango indexado:

```
db.usuarios.find({ edad: { $lt: 18 } });
```

MongoDB **no podrá usar el índice**, ya que los documentos con edad < 18 no fueron incluidos en él.

### Beneficios del índice parcial

- **Reduce el** tamaño del índice, ya que solo se incluyen documentos relevantes.
- Mejora el rendimiento de las consultas al evitar indexar documentos innecesarios.
- Acelera las operaciones de escritura, ya que MongoDB solo actualiza el índice cuando un documento cumple con la condición.
- Útil para casos donde **solo un subconjunto de la colección se consulta frecuentemente**.

## `background`: Creación de índices sin bloquear la base de datos

Por defecto, MongoDB crea los índices en **primer plano**, lo que puede bloquear operaciones de escritura temporalmente. Con **`"background": true`**, el índice se genera en segundo plano, permitiendo que la base de datos siga operando sin interrupciones (en versiones más recientes de MongoDB, esta opción es automática y no es necesario especificarla).

Por ejemplo:

```
db.usuarios.createIndex({ nombre: 1 }, { background: true });
```

## `collation`: Índices con reglas específicas de ordenamiento

La opción **`collation`** en MongoDB permite definir **reglas personalizadas de ordenamiento y comparación de texto**, lo que es especialmente útil en bases de datos multilingües o en casos donde se requiera una comparación que no distinga entre mayúsculas y minúsculas o que siga un orden específico según un idioma determinado.

Para definir un índice con una configuración de **`collation`**, se debe especificar el **idioma (`locale`)** y otras reglas de comparación:

```
db.productos.createIndex(
  { nombre: 1 },
  { collation: { locale: "es", strength: 2 } }
);
```

En este caso:

- **locale: "es"** → Define el idioma **español** para el índice, asegurando que el orden de los caracteres y reglas de acentuación sigan las normas del idioma.

- **strength: 2** → Hace que la comparación **ignore mayúsculas y acentos**, permitiendo que "Televisión", "Television", "televisión" y "television" sean tratados como equivalentes.

Por ejemplo, si tenemos los siguientes documentos en la colección productos:

```
{ "_id": 1, "nombre": "Televisor" }
{ "_id": 2, "nombre": "televisor" }
{ "_id": 3, "nombre": "Teléfono" }
{ "_id": 4, "nombre": "telefono" }
```

Una consulta que ignore mayúsculas y acentos se vería así:

```
db.productos.find({ nombre: "TELEVISOR" }).collation({ locale: "es", strength: 2 })
```

Este comando **devolverá ambos documentos** con "Televisor" y "televisor", ya que la comparación ignora diferencias de mayúsculas y minúsculas.

### Beneficios del uso de `collation`

- **Búsquedas insensibles a mayúsculas y acentos**, mejorando la experiencia del usuario.
- **Ordenación correcta según reglas lingüísticas**, útil en bases de datos con múltiples idiomas.
- **Compatible con índices**, permitiendo consultas rápidas sin afectar el rendimiento.

## Índices sobre texto

Los **índices sobre texto** en MongoDB están diseñados para facilitar y optimizar **búsquedas textuales** en documentos de una colección. A diferencia de los índices convencionales, que funcionan con valores exactos o rangos, los índices de texto permiten realizar búsquedas eficientes en **cadenas de texto**, identificando palabras clave dentro de uno o varios campos.

Las **búsquedas textuales** en MongoDB permiten localizar documentos que contienen ciertas palabras clave dentro de campos de tipo texto. A diferencia de las búsquedas tradicionales, que requieren coincidencias exactas en los valores de los campos, las búsquedas textuales **identifican palabras y frases dentro de textos más largos**, facilitando consultas más flexibles y avanzadas.

Este tipo de búsqueda es útil en aplicaciones donde los usuarios necesitan encontrar información relevante sin conocer exactamente cómo está almacenada en la base de datos, como en **motores de búsqueda, catálogos de productos, artículos de blogs o registros de comentarios**.

MongoDB optimiza estas búsquedas mediante los **índices de texto**, que permiten analizar y recuperar datos de manera eficiente. Gracias a estos índices, las consultas pueden incluir:

- **Palabras clave individuales** → Encuentra documentos que contienen una palabra específica.
- **Frases exactas** → Busca secuencias exactas de palabras en un campo.

- **Exclusión de términos** → Omite documentos que contengan palabras no deseadas.
- **Orden por relevancia** → Prioriza los documentos más relevantes según la consulta.

En MongoDB, los **índices sobre texto** se crean indicando "text" en lugar de 1 o -1 en la definición del índice. Mientras que los índices normales ordenan los valores en orden **ascendente (1) o descendente (-1)**, los índices textuales **no siguen un orden específico**, sino que permiten realizar **búsquedas de palabras clave dentro de los campos de tipo texto**.

Por ejemplo, para indexar un campo **`descripcion`** y habilitar búsquedas de texto en él, se define el índice así:

```
db.productos.createIndex({ descripcion: "text" });
```

Este índice permitirá realizar búsquedas eficientes dentro del campo **`descripcion`**, como en la siguiente consulta:

```
db.productos.find({ $text: { $search: "tecnología avanzada" } })
```

Aquí, MongoDB buscará documentos donde descripcion contenga las palabras "tecnología" o "avanzada", sin importar su posición dentro del texto ni si están en mayúsculas o minúsculas.

Para explicar el anterior párrafo, en MongoDB, las **búsquedas textuales** son **insensibles a mayúsculas y minúsculas**, lo que significa que buscar "MongoDB" es equivalente a buscar "mongodb", "MONGODB" o cualquier otra variante de capitalización. Esto facilita la búsqueda de términos sin importar cómo fueron escritos originalmente en la base de datos.

Además, MongoDB puede **ignorar ciertas palabras comunes** conocidas como **"stop words"**, dependiendo del idioma configurado en el índice de texto. En español, estas palabras suelen incluir **artículos (el, la, los, las), preposiciones (de, en, con, para), pronombres (yo, tú, él, nosotros), conjunciones (y, o, pero)** y otras palabras que aparecen con mucha frecuencia en el lenguaje natural pero que **no aportan un significado clave en la búsqueda**.

Por ejemplo, si tenemos un documento con el texto:

```
{ "descripcion": "El aprendizaje de bases de datos es fundamental" }
```

Y realizamos la siguiente búsqueda:

```
db.articulos.find({ $text: { $search: "aprendizaje bases datos" } });
```

MongoDB ignorará palabras como "El" y "de" y se enfocará en encontrar coincidencias con "aprendizaje", "bases" y "datos", mejorando la precisión y relevancia de los resultados.

Este comportamiento es útil para reducir ruido en las búsquedas, aunque en algunos casos podría ser necesario ajustar la configuración del idioma o deshabilitar la eliminación de **stop words** si queremos que todas las palabras sean consideradas

## Consideraciones finales sobre índices en MongoDB

El uso de índices en MongoDB es fundamental para mejorar el rendimiento de las consultas, pero no está exento de costos. Aunque los índices aceleran la recuperación de datos, también introducen una **sobrecarga en las operaciones de escritura**. Esto se debe a que cada vez que se inserta, actualiza o elimina un documento, MongoDB debe modificar los índices asociados para reflejar estos cambios.

En bases de datos con un alto volumen de escrituras, la presencia de múltiples índices puede ralentizar las operaciones y aumentar el consumo de recursos. Por ello, es importante evaluar cuidadosamente **cuántos y cuáles índices se deben crear**, asegurando que el beneficio en las consultas justifique el costo en rendimiento.

### Cuándo crear índices

Si bien los índices mejoran el rendimiento de las consultas, su uso debe estar bien justificado, ya que, como ya se ha mencionado, también afectan las operaciones de escritura y el consumo de almacenamiento. **No siempre es recomendable indexar todos los campos de una colección**, ya que cada índice introduce una carga adicional en las inserciones, actualizaciones y eliminaciones de documentos.

Se recomienda **crear índices solo cuando el beneficio en la velocidad de consulta supere el costo en rendimiento de escritura**. Esto es especialmente útil en bases de datos con un **alto volumen de lecturas (READs) y un bajo número de escrituras (UPDATEs y INSERTs)**, como en sistemas de reportes, análisis de datos o catálogos de productos.

Por el contrario, en colecciones con un **alto número de escrituras y pocas consultas**, la presencia de múltiples índices puede ralentizar significativamente el rendimiento, ya que cada nueva inserción o actualización implica modificaciones en los índices existentes.

Por ello, antes de crear un índice, es recomendable analizar:

- **Frecuencia de las consultas**: Si un campo se usa regularmente en búsquedas o filtrados, puede ser un buen candidato para un índice.
- **Impacto en las escrituras**: Si la colección recibe muchas actualizaciones o inserciones, demasiados índices pueden hacer que las operaciones sean más lentas.
- **Tamaño y almacenamiento**: Los índices ocupan espacio en disco y memoria RAM, por lo que es importante evaluar su impacto en los recursos del sistema.

Crear índices estratégicamente puede **mejorar el rendimiento sin comprometer la eficiencia** de la base de datos, asegurando que las consultas sean rápidas sin afectar excesivamente las operaciones de escritura.

### Otros tipos de índices en MongoDB

Además de los índices estándar que hemos visto, MongoDB ofrece otros tipos de índices especializados para necesidades específicas:

### Índices Hash

Los **índices hash** en MongoDB se utilizan para **optimizar búsquedas exactas** en campos específicos. A diferencia de los índices tradicionales, que permiten ordenar y realizar búsquedas por rangos, los índices hash generan un **valor hash único para cada valor del campo indexado**, distribuyendo los datos de manera uniforme en la base de datos.

Los índices hash tienen las siguientes características:

- Son ideales para búsquedas exactas, pero no permiten búsquedas por rangos ni ordenamientos.
- Cada valor del campo indexado se convierte en un **hash**, lo que garantiza una distribución equitativa de los datos.
- Se utilizan comúnmente en **bases de datos fragmentadas (sharded clusters)** para mejorar la distribución de los documentos entre nodos.

Por tanto, los **índices hash** son especialmente útiles en **búsquedas exactas**, donde se necesita encontrar coincidencias precisas en campos como **correos electrónicos, identificadores únicos o nombres de usuario**, sin requerir ordenamientos ni búsquedas por rangos. Además, desempeñan un papel clave en la **distribución de datos en bases de datos fragmentadas (sharded clusters)**, ya que permiten utilizar un **campo con índice hash como clave de partición**, asegurando que los documentos se repartan de manera uniforme entre los distintos nodos del clúster, evitando desequilibrios en la carga de los servidores.

Para definir un índice hash en un campo, se usa la opción "**`hashed`**" en la función **`createIndex`**():

```
db.usuarios.createIndex({ email: "hashed" });
```

Este índice optimizará las consultas que busquen un **email exacto**, como:

```
db.usuarios.find({ email: "usuario@email.com" });
```

Sin embargo, no será útil si intentamos **ordenar** o realizar búsquedas con operadores de comparación (**`$gt, $lt`**, etc.), ya que los valores se almacenan como hashes y no de forma secuencial.

### Índices para Geoposicionamiento

Los **índices geoespaciales** en MongoDB permiten almacenar y consultar **ubicaciones** basadas en coordenadas geográficas. Son utilizados en aplicaciones que necesitan buscar lugares cercanos, calcular distancias o encontrar puntos dentro de un área específica.

MongoDB ofrece dos tipos de índices geoespaciales:

- **Índice 2dsphere** → Se usa para datos en coordenadas reales de **latitud y longitud**, como mapas o aplicaciones de geolocalización.
- **Índice 2d** → Se utiliza para representar coordenadas en un **plano cartesiano**, útil en sistemas personalizados como videojuegos o simulaciones.

Estos índices permiten realizar búsquedas eficientes en bases de datos que manejan información de ubicaciones y distancias.

## Más sobre índices

Para obtener más información sobre los índices en MongoDB, su funcionamiento y las diferentes opciones disponibles, puedes consultar la documentación oficial en MongoDB Indexes. En esta guía, encontrarás detalles técnicos sobre la creación, administración y optimización de índices, así como ejemplos prácticos para aplicar en diferentes escenarios. Es un recurso valioso para profundizar en el uso de índices y mejorar el rendimiento de las consultas en bases de datos MongoDB.

URL: https://docs.mongodb.com/manual/indexes/